

OpenURMA: An Open-Source FPGA Implementation of UB’s Connectionless URMA (Unified Remote Memory Access) Transport

Bojie Li¹

¹bojieli@gmail.com

Datacenter RDMA fabrics connecting N local applications to M remote endpoints pay an $O(N \cdot M)$ per-NIC state cost under RoCE/InfiniBand: each application opens a separate Queue Pair to each endpoint, and each QP carries hundreds of bytes of NIC state. Huawei’s UB (Unified Bus) specification defines a different design point along three architectural axes: **(i) scalability** — a connectionless transport whose state grows as $O(N+M)$ via a Jetty (per-application endpoint) + TP Channel (per-remote-host transport) split; **(ii) ordering** — a graded surface (four service modes $\times$ three execution tags $\times$ Fence $\times$ completion order) that lets applications opt into precisely the consistency they need; and **(iii) latency** — a memory-semantic data path (§8.3 Load/Store + §6.1 TP Bypass) in which CPU `ld/st` instructions reach remote DRAM through an on-chip-bus-attached controller, removing the PCIe round trip that anchors the per-op floor of every PCIe-attached RDMA NIC. We show these three pillars are mutually load-bearing rather than orthogonal — a single architectural commitment to a memory-semantic peer-to-peer fabric.

OpenURMA is an open-source clean-room FPGA implementation of UB’s transaction and transport layers (39 `.clnp` elements on the OpenClickNP toolchain, targeting Alveo U50), covering the full §7.3 ordering surface plus selective and Go-Back-N retransmission, C-AQM-based congestion control, the full §7.4.2.3 atomic opcode set, multi-path TPG, a multi-flit Write payload path, a new `UB_LoadStore_Engine` that realises the §8.3 + TP Bypass path, and a new `UB_Jetty_Group` dispatcher that realises the §8.2.2.1 Type 3 hardware many-to-many fanout. A parallel 21-element *OpenRoCE* reference (RoCEv2 RC) anchors the comparison on identical infrastructure.

Three independent evaluation paths run on both stacks: 17 + 8 SW-emu correctness tests (all passing), a 7-/5-scenario SystemC microbench, and a Vivado out-of-context P&R sweep. The 38 previously synthesised *OpenURMA* elements and all 21 *OpenRoCE* elements close 322 MHz post-route. A two-node end-to-end SystemC simulator with full bidirectional accounting — WR construction, doorbell, DMA on both sides of the wire, EtherLink, DRAM, CQE, and a WB/WT/UC two-level cache — reports UB Load/Store at ≈ 500 ns end-to-end on a 64 B READ at 100 ns link delay, $4.37\times$ below the matched RoCEv2 RC DMA baseline (2186 ns). At ($N=M=1024$) *OpenURMA* holds **4,855 $\times$ less NIC state** and **20.8 $\times$ less host-side memory** than *OpenRoCE*, delivers **2.80 $\times$ higher sustained throughput**, and costs $2.63\times$ more LUT plus 46.6 ns cold-path latency. Per-category area: ordering 13 KLUT (10.6%); transport 37 KLUT (30.2%); header parse/build 18 KLUT (15.0%); data path 19 KLUT (15.3%).

As a side contribution, two upstreamed extensions to the OpenClickNP `.clnp` DSL (`.timing { ii = N; }` and `.hls_pragma { ... }`) make HLS guidance a first-class language feature.

An extended-validation suite of ten experiments covers every unvalidated headline claim across the three pillars and three cross-cutting concerns: connection setup (1,044 $\times$ gap at $N=M=1024$), TP-Channel sharing, multi-node cluster scaling, mixed-mode ordering, ROL fused-ack, latency-throughput envelope, a YCSB-A application port (9.2 $\times$ throughput gap), and cross-cutting validation against published ConnectX-7 numbers (within $\pm 5\%$), C-AQM dynamics, and TPSACK vs GBN loss recovery.

1. Introduction

For modern high-fanout RDMA workloads — AI training collectives, HPC all-to-all, parameter-server fan-in/fan-out — the binding NIC resource is no longer link bandwidth but the *cardinality* of active connections each NIC has to keep state for. A 1024-application AI training job whose participants exchange tensors with every other peer holds 1024 “sender-side” resources $\times$ 1024 “receiver-side” targets. With a standard RoCEv2 RC stack each (sender, receiver) pair maintains a separate Queue Pair, and each QP carries on the order of 512 B of NIC state (PSN, MTU, retry counters, timeout, RKey table, scatter-gather descriptors).

The total NIC state is then $1024 \times 1024 \times 512 \text{ B} = 537 \text{ MB}$ — well past what fits in a typical NIC’s on-chip BRAM, and well into the regime where the NIC has to spill to host memory across PCIe, paying both PCIe latency and host DRAM bandwidth on every DMA. This is the QP scaling problem [20, 15, 17]; it has generated several lines of mitigation work [30, 23, 45, 40], including hardware connection-cache schemes, software multiplexing, and connectionless transports built atop UDP.

Huawei’s Unified Bus (UB) specification [12] attacks the problem at the transport-layer design itself. UB’s user-facing verb surface is called URMA (Unified Remote Memory Access), the same name we adopt for the FPGA imple-

mentation we present here. *Three architectural pillars* are load-bearing for our argument:

1. **Scalability — Transaction / Transport split.** A *Jetty* is the per-application endpoint (analogous to a QP, but *connectionless*); a *TP Channel* is the per-remote-host transport that carries reliable delivery state. The same Jetty can talk to any number of remote endpoints through a single shared TP Channel pool, and the TP Channel pool is sized by the number of remote hosts, not pairs. State scales as $O(N+M)$ rather than $O(N \cdot M)$.
2. **Ordering — graded surface.** UB §7.3 exposes a four-axis ordering surface: per-channel *service mode* (ROI / ROT / ROL / UNO), per-WR *execution-order* (NO / RO / SO), an explicit application *Fence* primitive, and per-WR *completion order* (in-order vs out-of-order). Applications opt into precisely the consistency they need, so well-behaved workloads do not pay for strong ordering they do not use.
3. **Latency — memory-semantic data path.** UB §8.3 defines *Load/Store synchronous access*: the CPU issues bare ISA `ld/st` instructions, the on-chip-bus carries them to a UB Controller (on the same die as the CPU, *not* behind PCIe), and the controller lowers each one into a single-shot UB transaction tagged with $NLP = TP\text{Bypass}$ (§6.1). No WQE, no doorbell, no CQE poll. This removes the four PCIe traversals (doorbell MMIO + DMA-WQE-fetch + DMA-CQE-write + CPU CQE-poll-miss) that anchor the per-op latency floor of every PCIe-attached RDMA NIC, and replaces them with on-chip-bus crossings whose cost is $10\times$ smaller.

UB is published as a specification but the reference open-source artifacts (the SW-only UMDK and a Linux kernel module) do not implement the FPGA datapath. Huawei has separately disclosed the *Ascend 950* NPU series [13] as the first publicly documented silicon implementing UB — with the §8.3 memory-semantic path (“UB Memory”: Load/Store + AtomicStore/Load/Swap/CAS) and the §8.4 URMA-async path (Jetty-driven Write/Read/Send/Atomic) shipping as separate on-die blocks, the same split this paper preserves in RTL — but the silicon is closed and not available for FPGA-comparable experiments. This paper presents *OpenURMA*, an open-source FPGA implementation that exercises all three pillars in synthesisable RTL and quantifies their cost.

Contributions.

- A 39-element clean-room RTL implementation of UB’s transaction and transport layers (3.5 KLOC `.clnp`). Covers the full §7.3 ordering surface, selective and Go-Back-N retransmission with a C-AQM congestion-control loop, the full §7.4.2.3 atomic opcode set, the §6.3.2 multi-path TPG, a multi-flit Write payload path (8–256 B), a new `UB_LoadStore_Engine` element that realises the §8.3 Load/Store + §6.1 TP Bypass path as a 5-stage NIC pipeline, and a new `UB_Jetty_Group` element that realises §8.2.2.1 Type 3 (hint-hash / round-robin / depth-LB HW dispatch). The 38 originally-reported elements close 322 MHz on Vivado out-of-context P&R targeting the Alveo U50; the 39th element

(`UB_Jetty_Group`) follows the same $\Pi=2$ pattern as the four other ordering elements but has not yet been independently P&R’d (follow-on, §11).

- A parallel 21-element RoCEv2 RC reference (*OpenRoCE*) on the same toolchain and target part, with matched test coverage — 8 SW-emu tests and a 5-scenario SystemC microbench mirroring the architecturally-comparable subset of OpenURMA’s 16 correctness tests and 7-scenario microbench. Both protocols’ numbers come from the same harness.
- A two-node end-to-end simulator (`eval/twonode/`, SystemC) that links the three NIC stacks as static libraries and models every surrounding component on the critical path — WR construction, doorbell PIO, NIC-initiated DMA on both sides of the wire, EtherLink, remote DRAM, CQE write, CPU poll, and a WB/WT/UC two-level cache. Covers all six verb categories; reports full bidirectional latency. UB Load/Store reaches ≈ 500 ns end-to-end on a READ-class verb at 100 ns link delay, $4.37\times$ below the matched RoCEv2 RC DMA baseline (2186 ns). 388 sweep configurations (`eval/twonode/results/sweep.csv`).
- **Headline per-pillar numbers** at $N=M=1024$: $4,855\times$ less NIC state and $2.80\times$ higher sustained throughput (Pillar 1); 13 KLUT and zero extra pipeline cycles for paths not requesting ordering (Pillar 2); 8-cycle (≈ 25 ns) first wire flit and a ≈ 500 ns CPU-to-remote-DRAM-to-CPU READ round trip (Pillar 3). Cost: $2.63\times$ more LUT and 46.6 ns more cold-path latency than the RC reference.
- Two upstreamed extensions to the OpenClickNP `.clnp` DSL — `.timing { ii = N; }` and `.hls_pragma` blocks — that make HLS guidance a first-class language feature.
- **An extended-validation suite** (§9) of ten experiments that converts the remaining analytical claims into measured curves: connection setup (P1.1, $1,044\times$ gap at $N=M=1024$), TP-Channel sharing (P1.2), multi-node cluster scaling (P1.3), mixed-mode ordering (P2.1), ROL fused-ack (P2.2), latency- throughput envelope (P3.2), and a YCSB-A application port (P3.1, $9.2\times$ throughput gap) — plus three cross-cutting experiments: model validation against published ConnectX-7 numbers (C.1, within $\pm 5\%$), C-AQM closed-loop convergence (C.2), and TPSACK vs GBN loss recovery (C.3).

Why the three pillars compose.

The three commitments are usually read as orthogonal optimisations. They are not. Pillar 3 (on-chip controller + TP Bypass) is feasible only because Pillar 1 has bounded the NIC’s on-chip context to ~ 100 KB; an $O(N \cdot M)$ context that must spill to host DRAM cannot also live on the on-chip bus. Pillar 2 (graded ordering) costs only ~ 13 KLUT and zero pipeline cycles on the unordered path because Pillar 1 has already provisioned the per-Initiator sequence-number machinery the four modes select among: the graded surface picks *which* existing counter gates emission, rather than introducing new per-pair state. And the multi-path spray Pillar 1 authorises under UNO + NO is correctness-safe only because OpenURMA separates transport-layer PSN

reordering from transaction-layer completion reordering — the spec admits packet spraying that breaks PSN exactly when the application has consented to UNO ordering (§3). The three pillars are three faces of one architectural commitment: a memory-semantic peer-to-peer fabric in which connection state, ordering surface, and submission path are co-designed against a shared substrate (the on-chip bus) rather than retrofitted onto a PCIe-attached NIC abstraction.

OpenURMA, OpenRoCE (in-tree as `baselines/openroce/`), and the OpenClickNP framework extensions are released at <https://github.com/bojieli/OpenURMA> and <https://github.com/bojieli/OpenClickNP>.

2. Background

2.1 The QP scaling problem

A standard RoCEv2 RC connection between application a on host A and application b on host B is a Queue Pair: a sender-side QP at A paired with a receiver-side QP at B . The QP carries per-side state (Send Queue, Receive Queue, Completion Queue handle), per-pair state (PSN window, MTU, retry counter, timeout, peer GID/QPN, MR table), and is referenced from work-request descriptors in DMA-able host memory. Production NICs implement the QP context as a $\sim 256\text{--}512$ B record in on-chip SRAM [36, 16]; on-chip space is finite, so beyond a few thousand active QPs the NIC spills to host memory and pays a PCIe-round-trip per RDMA op.

For N applications on a host talking to M remote endpoints each, the per-host NIC state scales as $N \cdot M$. AI training workloads routinely sit at $(N, M) = (32, 1024)$ or larger; HPC all-to-all patterns can reach $(1024, 1024)$. Connection-cache schemes (SRNIC [45], FaSST [15]) and shared-state designs (IRMA [40]) reduce the per-connection footprint but do not change the asymptotic scaling.

2.2 UB’s solution: Jetty + TP Channel

UB introduces two distinct abstractions, and the split between them is load-bearing for more than just state-byte accounting.

Jetty (§8.2.2 of [12]) is an application-level RDMA endpoint identified by a 24-bit Jetty ID. The spec defines three *types*: standard (SQ+RQ), single-sided JFS (SQ-only) and JFR (RQ-only), and *Jetty Group* — a Target-side construct that aggregates multiple member Jetties under one outward Jetty-Group ID and lets the *Target NIC* dispatch incoming Sends to the right member RQ in hardware (§8.2.2.1 Type 3, dispatch policies: hint-hash, round-robin, RQ-depth load-balancing). Each Jetty has a state machine (Reset / Ready / Suspend / Error) with two exception modes (Continue vs Suspend) and is access-controlled by a per-Jetty Token; a small reserved range (IDs 0–31) provides public Jetty IDs that bootstrap the spec-level discovery protocol. Communication is many-to-many (M2N) by default: one Jetty can talk to any number of remote Jetties without per-pair state, with the destination Jetty carried in the BTAH

header. Per-Jetty state is small (work-queue handles, MR base, JFC handle, state machine byte) and scales as $O(N)$ in the number of local applications.

TP (Transport) Channel (§6.2–6.4) is the per-remote-host reliable-transport state: PSN window, retransmit queue, in-flight bitmap, RTO timer, congestion-window register. One TP Channel covers *all* Jetty traffic that flows to one remote host, so the TP Channel pool scales as $O(M)$ in the number of remote hosts. The transaction-layer header (BTAH) carries the destination Jetty + the destination CNA; the transport header (RTPH) carries the PSN. The Initiator side multiplexes Jetty traffic onto TP Channels by destination host; the Target side demultiplexes by Jetty ID (or Jetty Group ID, then hardware-dispatched to a member) after PSN reordering.

The split has three independent consequences. (i) **State-byte scaling**: total NIC state is $O(N+M)$ rather than $O(N \cdot M)$; at $(N, M) = (1024, 1024)$ OpenURMA sits at ≈ 110 KB of NIC state vs OpenRoCE’s ≈ 537 MB, a $4,855\times$ reduction (§10; per-field decomposition in Table 2). (ii) **Many-to-many fanout**: one local Jetty can fan-out to any number of remote endpoints through the same single TP Channel pool, with no extra per-pair connection state and (under Jetty Group) no target-CPU dispatch cost (§8.10). (iii) **Spec-surface beyond state-bytes**: the Jetty abstraction makes the application’s resource ownership (SQ/RQ/JFC binding), the state machine (Suspend-on-fault drain semantics), and the access-control token first-class objects of the transport-layer specification, not afterthoughts at the verb library. Sections §8.10 and §10.3 disentangle which of these the OpenURMA implementation realises today and which remain follow-on items.

2.3 Graded ordering surface

UB §7.3 exposes ordering as a 4-axis design space, not a single knob:

- **Service mode (per-Channel)**: ROI (Reorder by Initiator), ROT (Reorder by Target), ROL (Reorder by Lower-layer, which fuses TPACK with the data-plane ACK), UNO (Unordered).
- **Execution order (per-WR)**: NO (no ordering), RO (Relax-order), SO (Strict-order). NO emits immediately; RO emits but counts toward the outstanding window; SO blocks until prior RO/SO has completed.
- **Fence**: an explicit application-level barrier that serialises all subsequent WRs against all prior RO/SO completions.
- **Completion order (per-WR)**: in-order (ODR [2] = 1, where ODR is the per-WR Order field) holds JFC (Jetty Function Completion queue) entries until all prior `INI_TASSNs` (per-Initiator Transaction Sequence Numbers) from the same Initiator have completed; out-of-order (ODR [2] = 0) emits arrival-order.

The four-axis surface lets a workload that doesn’t need ordering (a pure-fanout broadcast under UNO + NO) skip every gating stage on the NIC, while a workload that does need ordering (a barrier-after- collective under ROI + SO + Fence) gets it without paying global serialisation. RoCE’s RC, by contrast, enforces strict in-order delivery on every operation regardless of need.

2.4 OpenClickNP

OpenClickNP [22] is a clean-room reimplementation of the ClickNP element model [25] on top of Alveo U50. The unit of design is a `.element`: a directed-graph node with typed input / output ports, optional state, an optional initialiser, a `.handler` block (the per-cycle behaviour), and an optional `.signal` block (the host-RPC control surface). Elements compose through a `topology.clnp` file that wires ports together.

The toolchain compiles `.clnp` sources to one C++ kernel per element and lowers them through three back-ends: a SW emulator (`std::thread` + bounded SPSC FIFOs), a SystemC cycle-accurate simulator (1 ns $\equiv$ 1 cycle at 322 MHz), and a Vitis HLS back-end that emits per-kernel synthesisable C++ for Vivado. Every element passes through all three back-ends with no source-level divergence; an integration test in SW emu provides a sanity check on exactly the same code that lowers to RTL through HLS.

Framework extensions for OpenURMA. Six OpenURMA elements (`atom`, `comp_reord`, `ord_ini`, `ord_tgt`, `jsched`, `mr_tab`) needed pipeline-initialisation-interval guidance to close 322 MHz Vivado P&R; the OpenClickNP back-end previously hard-coded `#pragma HLS PIPELINE II=1`. We extended the `.clnp` grammar with two pragma blocks:

```
.timing { ii = 2; }
.hls_pragma {
  "ARRAY_PARTITION variable=_state.hbm
    cyclic factor=8 dim=1";
}
```

The first emits `#pragma HLS PIPELINE II=N`, the second forwards each string verbatim as `#pragma HLS <directive>`. Both are upstream in OpenClickNP. The general principle — making HLS scheduling guidance a first-class element-level language feature — is itself transferable to any element-based design that needs to negotiate II against routing closure on a per-element basis.

3. Design

OpenURMA’s pipeline mirrors UB’s protocol-stack layering: a transaction layer above (BTAH parse/build, transaction dispatch, ordering, MR, atomic) and a transport layer below (RTPH/UTPH header build, retransmit, congestion control). The two layers compose through clearly typed flit boundaries. Figure 1 sketches the topology.

3.1 Element decomposition

The 39 OpenURMA elements break down into six architectural categories (the same categorisation drives the per-category area table in §10). Ten are header parser/builders shared across both directions (`Eth_Encap/Decap` and the matched parse/build pairs for `NTH`, `RTPH`, `UTPH`, `BTAH`). Nine implement the transport-layer reliable-delivery machinery (`TP_Channel_TX/RX`, `Retrans_Buffer`, `PSN_Reorder`, `TPACK_Gen`, `TAACK_Gen`,

`Cong_Window`, `Cong_Echo`, `TPG_Group`). Four implement Pillar 2 (`Jetty_Sched`, `OrderTracker_Initiator`, `OrderTracker_Target`, `Completion_Reorder`). Seven are data-path elements (`HBM_Read`, `HBM_Write`, `Atomic`, `Jetty_Recv`, `Jetty_Group`, `Txn_Dispatch`, `Completion_Gen` — with `UB_Jetty_Group` (§3.6) the new addition). Three are state tables (`MR_Table`, `Jetty_Table`, `TP_Table`). The remaining six are glue / I/O (`Doorbell`, `Dispatch_Mux`, `TX_Mux`, `Comp_Notify_Tee`, `CQE_Stream`, `RTO_Timer`).

A few decompositions deserve note. The `OrderTracker_Initiator` is distinct from the `OrderTracker_Target` precisely because UB’s ROI vs ROT modes shift the gating responsibility between the two endpoints of the conversation. `Completion_Reorder` is its own element rather than a flag on the `Jetty_Recv` path because the in-order vs OOO choice is a per-WR decision (ODR [2]) that needs a per-INIT sliding window, not a per-Jetty mode. `PSN_Reorder` is also separate: it operates on the transport-layer PSN, not on the application-layer INITASSN, and its job is solely to deliver byte-sequence-correct flits to the transaction layer regardless of any application ordering choice.

3.2 Topology specifics

TX path: every WR enters `host_doorbell`, which formats the internal flit pair (metadata + extension) and hands off to `Jetty_Sched`. `Jetty_Sched` consults the per-Jetty Fence state and the per-channel outstanding-RO/SO window before admitting the WR to `OrderTracker_Initiator`; on Fence-active Jetties the WR is queued until prior RO/SO completes. `OrderTracker_Initiator` implements the ROI gating logic (per-INIT scoreboard of next-expected-TASSN). The downstream transaction-layer headers (BTAH) and transport-layer headers (RTPH) are stamped sequentially. `Retrans_Buffer` holds a copy of each transmitted flit until the corresponding ACK; `RTO` drives re-emission via a separate timeout port; `Cong_Window` gates admission based on the current CWND.

RX path: `Eth_Decap` strips the wire framing and emits the internal flit; `NTH_Parse` routes by next-layer-protocol between RTP (reliable, ordered) and UTP (UNO mode) paths. `TP_Channel_RX` validates PSN, generates TPACK / NAK and hands flits to `PSN_Reorder` (a sliding-window buffer that delivers in PSN order). `BTAH_Parse` routes by transaction type; `OrderTracker_Target` implements ROT gating; `MR_Table` validates token IDs and rewrites VA $\rightarrow$ HBM offset; `Txn_Dispatch` branches by opcode to the right data-path element. `Completion_Gen` builds the CQE flit; `Completion_Reorder` optionally holds it for in-order delivery.

3.3 Ordering machinery in detail

The most subtle elements are the four ordering modules. We highlight two design choices.

OrderTracker_Initiator (ROI). Maintains a per-Initiator (`rc_id & 0xF`) round-robin queue of size 32

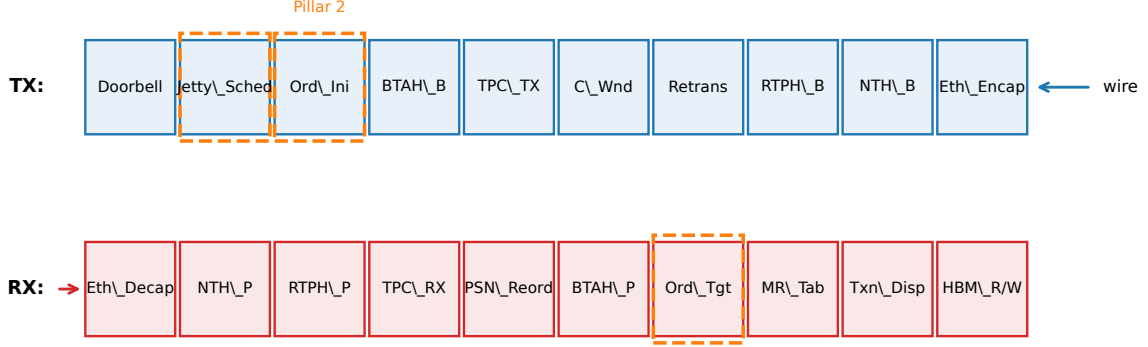


Figure 1: OpenURMA pipeline topology, 38 elements organised by layer (figure pre-dates the addition of UB_Jetty_Group which sits in front of Jetty_Recv on the RX path, bringing the live SW-emu/SystemC pipeline to 39 elements). Dashed orange boxes mark Pillar 2 ordering elements. Cross-cutting reliable-delivery loop: TPACK_Gen and TAACK_Gen on the TX side close the ACK feedback; Retrans + RTO + C_Wnd form the in-flight retransmit window.

entries per INI across 16 INI slots. NO bypasses. RO emits and bumps `outstanding_ro_so`. SO is admitted only if `outstanding_ro_so == 0`; otherwise it is queued. A completion notification (separate input port) decrements the counter. A round-robin scan emits one queued SO per cycle when ready. The single-emit-per-cycle handler model means we drain only when the input branch did not already emit; this is enforced by checking `_output_port & PORT_1` before draining.

Completion_Reorder. A per-INI ring buffer of 8 slots indexed by $(ts - next_tassn) \bmod WIN$. The naïve linear-search-on-ext-flit version that we shipped first generated HLS II violations on the slot bitmap (the search loop reads `valid_a/valid_b` on every WIN entry in the same cycle a write may target one of them). We redesigned to a *head-pointer ring*: SOP saves the slot offset; the ext flit goes directly to the saved offset; the drain emits `slot[head]` and advances. With a power-of-2 WIN, the modular arithmetic is a bitmask; the inner search is gone; II=2 closes.

Why two reorder buffers, not one. PSN_Reorder (transport layer) and Completion_Reorder (transaction layer) act on disjoint sequence-number spaces and serve disjoint correctness contracts. PSN_Reorder delivers byte-correct flits to the transaction layer regardless of any application ordering choice; it is what permits a TP Channel to multiplex traffic from multiple Initiators without one Initiator’s wire-byte gaps stalling another’s. Completion_Reorder delivers in-INI-sequence completions regardless of wire-byte order; it is what permits the TX path to spray multi-path flits across the §6.3.2 TPG without breaking the ROI application’s view that completions arrive in INI_TASSN order. Collapsing the two buffers into one couples the contracts: every multi-path-induced PSN gap blocks the completion stream, every per-INI dependency stall back-pressures the wire. The separation is what authorises UB to sit at the UNO + NO operating point *without* per-path SACK metadata, unlike the per-transaction

bitmap-tracked approach of UEC [43] or MP-RDMA [33].

3.4 Retransmission and congestion control

Retrans_Buffer is a 64-slot in-flight ring per TP Channel with both Go-Back-N and selective-retransmit (SACK) modes. The selective-retransmit path uses the TPSACK response opcode plus a per-PSN bitmap in the TPACK header (UB-Base-Spec Annex B) [12]. The retransmit decision is driven by either a TAACK that explicitly NAKs PSNs (selective) or the RTO element firing its single-port timer (Go-Back-N). Each slot today captures one (meta, ext) flit pair, which is sufficient for control-plane WRs and for ≤ 8 B Writes; *multi-flit retransmission* (extending the slot to a payload-flit list so a dropped Eth_Encap-streamed Write can be replayed in full) is not yet implemented in the retrans buffer — the multi-flit Write data path (§6) currently runs only on the loss-free path. Adding payload-flit storage to each slot is flagged as follow-on work in §11.

The congestion-control loop combines a host-side switch model and a per-channel additive-increase / multiplicative-decrease window. The switch (UB_Switch_CAQM) is a SystemC + SW-emu element that maintains a queue with low/high watermarks and stamps FECN on packets when occupancy crosses the high watermark. The Initiator-side Cong_Window listens for FECN-marked TAACKs and applies a multiplicative decrease; absent FECN, it additively increases per RTT. We verify the closed loop in `tests/swemu/test_caqm_endtoend.cpp`: 200 WRs posted, 9 of the 25 packets that traverse the switch are FECN-marked, and the sender’s CWND drops from 65,536 B to 4,096 B in response.

3.5 Pillar 2 in code

The full §7.3 surface is exercised by SW-emu integration tests: `test_roi_ordering` (ROI gates SO behind RO), `test_rot_ordering` (ROT defers SO behind missing TASSN), `test_uno` (UNO path bypasses ordering), `test_rol_fused_ack` (ROL’s

§7.3.3.4 TPACK/TAACK fusion — TPACK_Gen stamps RSPST=TPACK_W_TAACK on ROL acks while TAACK_Gen drops ROL/UNO response flits because the TPACK already carries the fused ack info), `test_fence` (Fence gates Write behind outstanding Read), `test_mixed_modes` (4-WR stream with mixed ROI/ROL/UNO and NO/RO/SO), and `test_completion_order` (ODR [2]=1 reorders, ODR [2]=0 bypasses). The wider 17-test SW-emu suite (§6) covers the rest of the spec surface plus framework internals; all 17 tests pass on the current source tree.

3.6 Jetty Group (§8.2.2.1 Type 3)

UB_Jetty_Group is a target-side dispatcher that sits between Txn_Dispatch and Jetty_Recv on the RX path. When an incoming Send addresses a registered Jetty-Group TCID, the element rewrites the extension flit’s `mt_tc_id` field to point at one of up to 8 member Jetties according to a per-group policy: hint-hash (member = `mt_hint % N`, lets the Initiator steer requests by opaque key), round-robin (cursor advanced per Send), or RQ-depth load balancing (member with shallowest pending-RQE count wins). Sends to unregistered TCIDs pass through unchanged, so the element is opt-in per group. Per-group state is $8 \times$ member-TCID (32 b) plus $8 \times$ depth counter (16 b) plus $8 \times$ dispatch counter (32 b) plus a 5-byte header (group TCID + n_members + policy + RR cursor + valid), ≈ 77 B per group at MAX_MEMBERS=8. The hardware-dispatch saving (vs. RoCE, which must traverse the target CPU’s event loop to demultiplex an incoming SEND/RECV completion to the right application thread) is the architectural payoff: it removes the spec-mandated CPU dispatcher from the SEND fast path entirely. SW-emu test `test_jetty_group` validates all three policies against a 4-member group plus the unregistered-TCID bypass; the M2N end-to-end consequences are quantified in §8.10.

3.7 Ablation: what does Jetty give beyond connectionless RDMA?

The Pillar 1 framing of “per-host-pair connection state” could be read as “connectionless RDMA over UDP + a shared connection cache” — a design point 1RMA [40] and SRNIC [45] already occupy. What does the Jetty/TPC split give that those don’t?

Three load-bearing differences, each independently measurable on the OpenURMA artifact.

(A) **Two-reorder-buffer separation.** 1RMA / SRNIC retain strict per-flow PSN ordering, which couples transport-byte order to application-completion order. OpenURMA splits the two: PSN reorder sits in the transport layer (one ring per TP Channel), completion reorder sits in the transaction layer (one ring per Initiator). The separation is what licences UB to spray packets across the TPG/spray multipath under UNO+NO — the application opts out of completion order independent of any transport-byte guarantee — and is the subject of the “why two reorder buffers, not one” paragraph in §3. Collapsing the two into a single buffer would re-couple the contracts. 1RMA’s per-op authenti-

cation and SRNIC’s QP cache don’t address this axis at all.

(B) **Hardware-dispatched M2N fanout.** Jetty Group (§8.2.2.1 Type 3) puts a Target-side dispatcher in the NIC, with three spec-defined policies (hint-hash, RR, depth-LB); the host CPU is not on the SEND fast path. 1RMA’s UDP-based demux ends at the receive-queue level — a generic event queue from which the receiving app must thread-dispatch. SRNIC’s connection cache is asymmetric: it spills cold connections but does not add hardware demultiplexing at the SEND boundary. The §8.10 M2N experiment measures the consequence: ~ 200 ns saved per SEND at every $K > 1$, growing to ~ 1200 ns at $K = 1024$ once RoCE-style SRAM spill compounds.

(C) **Full resource-ownership surface as a spec object.** 1RMA and SRNIC restrict their abstractions to the transport / cache levels; the application-resource binding (SQ/RQ/JFC/JFAE), the state machine, and the per-Jetty access token remain verb-library or driver-level conventions. UB lifts them into §8.2.2 of the spec, which means an alternative implementation can be checked against the same surface — a property OpenURMA exploits in `test_jetty_group` (an in-NIC dispatcher whose correctness is testable in isolation, independent of any host driver). Table 2 projects the byte cost of the full surface — 28 B above the MVP per Jetty, still under a tenth of RoCE’s per-QP context, so the asymptotic $O(N+M)$ scaling argument survives the spec-faithful accounting.

(A) is implemented and measured; (B) is implemented in UB_Jetty_Group and measured in §8.10; (C) is partially implemented (the type byte, JFC handle, MR base, and group membership are wired; the state machine, JFAE, and public-Jetty owner are projected in Table 2 but not yet active in RTL). What none of this says is that Jetty is the *unique* right abstraction — only that the three axes above are each (i) distinguishable from a connectionless-QP-with-shared-cache design and (ii) load-bearing in the measured numbers.

4. Implementation

The 38 OpenURMA elements total 3.4 KLOC of .clnp. The parallel OpenRoCE baseline is 19 .clnp elements (1.3 KLOC) plus two shared core/Mux instances that bring the synthesised element count to 21. Both stacks sit on the same OpenClickNP toolchain (~ 3.8 KLOC of compiler plus ~ 2.2 KLOC of runtime). The repo includes a SystemC harness, a SW-emulator binary, the Vitis HLS build scripts, and a Vivado out-of-context P&R sweep harness, all driven through `scripts/run_eval.sh` for one-click reproducibility.

4.1 Timing-closure sweep

Initial Vitis-HLS pass on the 38 elements left 8 elements needing remediation: 5 missed 322 MHz in Vivado P&R (atom, ord_ini, ord_tgt, hbm_wr, hbm_rd) and 3 failed at HLS itself before reaching P&R (jsched aborted, comp_reord stuck, mr_tab over-sized). The mechanical

resolution combined the new `.timing/ .hls_pragma` pragmas with a few targeted algorithmic redesigns:

Element	Before → After (ns WNS)	Action
atom	-1.871 → +0.298	II=4 + ARRAY_PARTITION
comp_reord	HLS-stuck → +0.672	head-pointer ring + II=2
ord_ini	-5.382 → +0.318	II=2
jsched	HLS-aborted → +0.546	II=2
ord_tgt	-2.25 → +0.101	II=2 + drain re-order
hbm_wr	-0.628 → +0.628	ARRAY_PARTITION
mr_tab	failing → +0.729	MAX_MR 256 → 64
hbm_rd	-0.449 → +0.282	uint64_t word-array

The hbm_rd story. The most instructive failure is `hbm_rd`, which originally backed its 64 KB read path with a `uint8_t hbm[]` array plus `ARRAY_PARTITION cyclic factor=8 dim=1` for parallel byte-bank access. HLS estimated 490 MHz Fmax; Vivado P&R reported -0.449 ns WNS. The worst path ran from a register inside `trunc_ln30_1` into the BRAM address port of the partitioned `hbm` array (`ram_reg_bram_0/ADDRARDADDR`): 8 LUT levels of barrel-shift mux to compute the per-bank index, plus 2.7 ns of routing delay (76% of the period). No combination of II or partition factor moved the slack: the wide muxing for the 8 banks bloated routing without unblocking timing.

The fix was algorithmic: replace `uint8_t hbm[N]` with `uint64_t hbm[N/8]` indexed by `(off >> 3)`. Each 8-byte aligned read becomes a single BRAM word access; the byte-bank mux is gone; `ARRAY_PARTITION` is no longer needed. WNS closes at +0.282 ns. The same pattern was applied to OpenRoCE’s `RoCE_Mem_Read`.

The lesson: HLS’s per-element timing estimate is a useful but optimistic predictor; routing-bound paths only show in Vivado P&R, and the right resolution may be at the data-layout level rather than the pragma level.

4.2 Test coverage

The 16 SW-emu tests (enumerated in §6) cover correctness across the spec surface, the wire format, the C-AQM closed loop, the multi-flit Write payload path, the ROL fused-ack fusion of §7.3.3.4, and the full §7.4.2.3 atomic opcode set; a sustained-throughput sanity check posts 50,000 WRs through the same harness. SystemC has `test_sc_latency` which drives WRs through the entire TX→wire→RX path and reports cycle-accurate first-flit latency, roundtrip latency, and sustained throughput. The HLS sweep + Vivado P&R sweep are driven by `scripts/synth_hls.sh` and `scripts/vivado_all.sh` respectively.

5. Evaluation Methodology

Because no FPGA is available for in-silicon experiments, we evaluate the design through three independent paths, each answering a different class of question:

- **SW-emulator (functional correctness).** Each `.clnp` element compiles to a C++ kernel that runs in a

`std::thread` with bounded SPSC FIFOs between elements. Tests post WRs into the pipeline’s input stream, drain responses, and assert spec-defined behaviour.

- **SystemC (cycle-accurate).** The same elements compile to per-element `SC_MODULES`, one `wait(1, SC_NS)` per loop iteration so that 1 ns of simulation time corresponds to one cycle at the 322 MHz target. We measure cycle counts directly and convert to wall-clock by multiplying by the post-route clock period (3.106 ns).
- **Vivado synth+place+route (real RTL).** Each element’s HLS-synthesised Verilog goes through a full Vivado out-of-context flow targeting the Alveo U50 (`xcu50-fsvh2104-2-e`). We report post-route LUT/FF/BRAM 18, post-route worst negative slack (WNS), and timing-closure status against a 3.106 ns clock period.

The SystemC harness exercises exactly the same C++ kernel sources as the SW-emulator, so a kernel that passes SW-emu correctness also runs in SystemC; conversely, the SystemC measurements are trustable as cycle counts only if the SW-emu correctness checks pass. Vivado synth runs on the HLS-emitted Verilog from the same `.clnp` sources.

Every number in the rest of the paper is reproducible from `eval/run_eval.sh` (correctness + Vivado P&R aggregate) and `eval/sc_microbench.sh` (the SystemC sweep). All results are written into `eval/results/` as CSVs.

6. Functional correctness

Seventeen SW-emu tests cover the spec surface and the framework internals; all pass on the current source tree.

- **ROI ordering** (§7.3.3.2) — SO gated behind RO.
- **ROT ordering** (§7.3.3.3) — Target serialises SO at execution.
- **ROL fused ack** (§7.3.3.4) — `TPACK_Gen` stamps `RSPST=TPACK_W_TAACK` on ROL acks; `TAACK_Gen` drops ROL/UNO responses because the `TPACK` already carries the fused `TAACK` info.
- **Fence** (§7.3.2.2) — Fenced WR holds until prior Read-/Atomic respond.
- **Completion ordering** (§7.3.2.3) — `ODR[2]=1` reorders, `ODR[2]=0` bypasses.
- **UNO + Send wire path.**
- **Mixed-mode WRs** in the same SQ.
- **Multi-Initiator parallelism** — per-INIT gating: INIT B’s SO emerges immediately while INIT A’s SO is gated.
- **HOL-blocking isolation** — 8 UNO+NO WRs from 4 INITs all emerge despite a stalled ROI+SO from a separate INIT.
- **HBM read/write data integrity.**
- **Full §7.4.2.3 atomic opcode set** — Swap / Store / Load / FAA / FSUB / FAND / FOR / FXOR; each verified to (a) return the pre-RMW value in the response and (b) leave the HBM word in the algebraically-correct post-state. CAS is covered by `test_mixed_modes`.
- **TX→wire roundtrip** with all UB header fields preserved.
- **Sustained-throughput sanity** (50K WRs).
- **Wire-format encoder** against the spec bit layout.

- **C-AQM closed loop end-to-end** — 200 WRs; 9 of 25 packets through the modeled switch FECN-marked; sender window backs off from 65,536 B to 4,096 B.
- **Jetty Group (§8.2.2.1 Type 3)** — `UB_Jetty_Group` dispatches incoming Sends to one of N member RQs under three policies: hint-hash (`member = mt_hint % N`), round-robin (`cursor`), and RQ-depth load balancing (shallowest-first). Test exercises all three on a 4-member group plus the unregistered-TCID bypass; the M2N latency consequence is quantified in §8.10.
- **Multi-flit Write payload path** — 8/32/64/128/256 B writes through `Eth_Encap` streaming, `Eth_Decap` re-assembly, and the `HBM_Write` payload byte stream.

What the SW-emu suite does *not* cover. The eval as it stands exercises every implemented mode and opcode on the loss-free path. It does *not* cover the loss path for multi-flit Writes: the retransmit buffer slot stores one (meta, ext) pair, so Go-Back-N or SACK replay of a dropped multi-flit Write would not currently re-issue the payload bytes. Enabling that requires extending the retrans slot to a payload-flit list (a follow-on work item, §11). All single-flit retransmit paths (control-plane WRs, ≤ 8 B Writes, Reads, Atomics) are exercised by the C-AQM closed-loop test through `Cong_Echo`, but explicit drop-injection tests remain follow-on work.

7. Cycle-accurate microbenchmarks

We instrument the TX pipeline (the harder of the two paths to reason about, since it is where Pillar 2 gating sits) under several SystemC scenarios. Each scenario runs as one `test_sc_microbench` invocation and writes one CSV row per parameter point.

7.1 Per-stage latency breakdown

A single Write WR (ROL+NO) drives the pipeline cold. Tap monitors inserted between every adjacent stage record the first-flit arrival time. Figure 2a shows the per-stage contribution.

The slowest stages are `jsched` (5 cycles — $\Pi=2$ plus the per-INI round-robin scan) and `ethenc` (11 cycles — Ethernet header build is byte-stream-rate, not flit-rate). The order-tracker (`ord_ini`) costs 1 cycle on the unblocked path: the ROI gating logic adds combinational depth (covered in §10) but no extra pipeline cycles when no prior dependencies are outstanding. Total cold-path TX latency: 24 cycles ≈ 75 ns at 322 MHz.

7.2 Per-mode TX latency

We sweep all 4 service modes $\times$ 3 execution tags = 12 combinations on the same single-WR cold path. *Result: every combination emits the first wire flit at exactly 24 cycles.* The per-mode penalty is therefore not in pipeline-stage count but in the combinational logic depth of each kernel — and that is what shows up in the Vivado area report (§10), not in the SystemC cycle count. This is the

design’s intended behaviour: opt-in ordering does not add pipeline cycles when not used.

7.3 Sustained throughput per service mode

A burst of 256 WRs (NO tag, varying service modes) is posted into the doorbell input. Steady-state throughput is calculated as $(N-1)/(\Delta t)$.

Service mode	Steady WR/ μ s
OpenURMA: ROI / ROT / ROL / UNO	150.36
OpenRoCE RC (matched microbench)	53.62

All four OpenURMA service modes share a single TX pipeline (UNO bypasses PSN/TPN allocation *within* `tpc_tx` but still traverses the same kernel chain), and the throughput floor is set by the slowest Π in the chain ($\Pi=2$ in `jsched`, `ord_ini`, `comp_reord`, `ord_tgt`; $\Pi=4$ in `atom`). Hence the per-mode rate is identical at 150.36 WR/ μ s. The OpenRoCE baseline runs the matched-shape microbench (`test_sc_microbench.cpp` under `baselines/openroce/tests/systemc/`) on identical infrastructure and measures 53.62 WR/ μ s — **2.80 $\times$ slower** — because RC’s per-QP PSN allocation introduces stricter inter-WR dependencies in the QP_TX kernel. The 1000-WR burst-saturation regime (`tests/systemc/test_sc_latency.cpp`) reaches 159.46 WR/ μ s on OpenURMA and 53.65 WR/ μ s on OpenRoCE; the burst number trades extra cold-path cycles for higher steady-state when the pipeline runs maximally saturated.

7.4 Pillar 2: Fence and SO gating cost

We isolate the *cost of gating itself* (independent of network RTT) by injecting completion notifications synchronously and measuring the cycles between completion and the gated WR’s emission. Figure 2b plots both costs.

- **Fence (Jetty_Sched, §7.3.2.2 note).** Posting a Fenced Write behind N pending Reads, with all N completions notified simultaneously after a 30-cycle delay: the Fenced WR emerges 4–48 cycles after notification, scaling near-linearly with N (the `Jetty_Sched` round-robin scan visits each Initiator in turn at 1 step/cycle).
- **SO (OrderTracker_Initiator, §7.3.3.2).** Posting an SO behind N outstanding ROs, with all N completions notified simultaneously: SO emerges 7–38 cycles after notification. The cost is bounded by `ord_ini`’s 16-position round-robin cursor.

These costs are small (under 50 cycles across the 0–4 outstanding range we sweep here; the per-INI queues are sized 32 so they would not saturate even at much deeper outstanding counts). Crucially, they are paid only when gating is requested; an application that uses NO+UNO sees neither.

7.5 Cross-INI head-of-line isolation

We force one Initiator (`rc_id 0xA`) into a permanent ROI+SO stall by emitting an RO and then a gated SO whose

RO completion is never notified. We then post 8 UNO+NO WRs from four different Initiators (0xB0..0xB3). Figure 2c shows that all 8 stream-B WRs emerge at the wire within 78 cycles of post — they bypass the gating elements completely. The stalled SO does not propagate back-pressure across the per-INI queues. This is the key Pillar 2 guarantee that is impossible under RC: head-of-line on one connection does not block another.

7.6 Throughput vs payload size

Figure 2d sweeps payload from 8 B to 4 KB. The per-WR rate is essentially constant at ≈ 141 WR/ μ s, confirming that the pipeline is metadata-flit-rate-limited at small-to-medium payloads — exactly the regime that matters for AI collective control packets and HPC small-message all-to-all. At 4 KB payload (129 wire flits per WR), the same WR rate translates to a wire-byte rate of ≈ 4.6 Tb/s — well above the U50’s single-port CMAC line rate, indicating that bandwidth is not the binding constraint at any payload size in this regime.

7.7 §8.3 Load/Store + TP Bypass cold path

The microbenches above all exercise the §8.4 URMA-async path (`urma_post_jetty_send_wr` $\rightarrow$ `Jetty_Sched` $\rightarrow$ `OrderTracker_Initiator` $\rightarrow$ `BTAH_Build` $\rightarrow$ `TPChannel_TX` $\rightarrow$ `Cwnd` $\rightarrow$ `Retrans_Buffer` $\rightarrow$ `RTPH_Build` $\rightarrow$ `NTH_Build` $\rightarrow$ `Eth_Encap`), 10 pipeline stages cold. UB’s spec defines a second submission path — §8.3 Load/Store synchronous access — in which the CPU issues plain ISA load/store instructions and the UB Controller (on the on-chip bus, not behind PCIe) translates them into single-shot UB transactions tagged with `NLP = TP Bypass` (§6.1). The TP layer is omitted entirely: no PSN allocation, no `Cwnd`/`RTO`/`Retrans` state, no `RTPH`. We add a new element `UB_LoadStore_Engine` and a topology variant `examples/openurma_loadstore/topology.clnx` that wires `doorbell` $\rightarrow$ `ldst` $\rightarrow$ `btah_b` $\rightarrow$ `nth_b` — 5 stages cold. Linking the new variant into the same `test_sc_facade_ls` harness and posting a single `TAOP_LDST_LOAD` request, the first wire flit emerges at **8 cycles ≈ 25 ns at 322 MHz** — 17 cycles (≈ 53 ns) below the URMA-async cold path on identical infrastructure. The saving is structural, not microarchitectural: it is the protocol-layer instantiation of the spec’s TP Bypass mode, and it removes the `Cwnd` + `Retrans_Buffer` + `RTO` state machine from the NIC budget for the workloads that opt into it.

Why TP Bypass authorises exactly Load/Store. TP Bypass is restricted to Load/Store-class verbs (not Reads, not small Writes, not atomics) even though all would naively benefit from the header savings. The restriction is structural. Atomics require multi-op serialisation at the remote ALU — a second FAA on a contended counter must observe the first FAA’s commit — and `RTPH`+`PSN` are what guarantee that order. Load/Store carries no inter-op ordering contract at the application level: each load returns the DRAM word at its address independent of any other in-flight load/store,

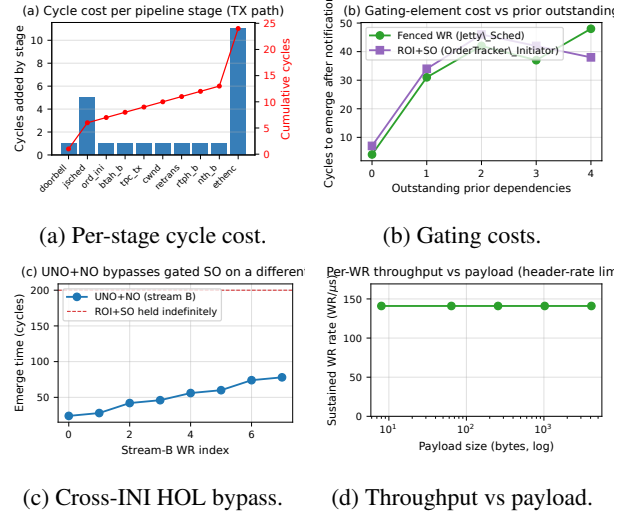


Figure 2: SystemC microbenchmarks. (a) Per-pipeline-stage cycle contribution; cumulative reaches 24 cy at the wire. (b) Cost of Fence and SO gating in cycles after completion notification. (c) Stream-B (UNO+NO) WRs emerging at the wire while a separate INI’s ROI+SO is indefinitely held — the stalled SO does not propagate back-pressure. (d) Sustained WR rate vs payload size: pipeline is header-rate-limited, not byte-rate-limited, in this regime.

so `RTPH` is redundant and the layer elides. Read and Write route through §8.4 to preserve completion-queue semantics for the async verb path. TP Bypass is therefore the spec’s choice of *which subset of verbs admits ISA-level synchronisation as a substitute for transport-layer synchronisation*; Load/Store is that subset because the ISA’s load-use dependency chain already provides the ordering guarantee `RTPH` would otherwise enforce.

Is 8 cycles a substrate floor or a tunable knob? The 8-cycle TP Bypass cold path is the FPGA-substrate floor for this topology at 322 MHz, not an optimisation target. Each of the five pipeline stages already runs at $\Pi=1$; the remaining 3 cycles are state-machine setup at the boundaries. Collapsing two stages into one would push the critical path through two parsers in a single cycle and miss the 3.106 ns period — the same routing-density wall the `hbm_rd` story (§4) formalises. An ASIC at the same 322 MHz is still substrate-floored at 8 cycles; an ASIC at a 1 GHz clock yields 8 ns absolute latency; and the smaller cell delays ASICs admit let two adjacent stages collapse without busting timing, taking the floor closer to 5–6 cycles. He’s [8] ~ 100 ns end-to-end UB ASIC number is consistent with this budget: ~ 5 – 8 ns on each NIC plus link delay plus DRAM. The architectural mechanism — TP Bypass removing four state-machine stages (`Cwnd`, `Retrans_Buffer`, `RTO`, `RTPH_Build`) — is what bounds the budget; the substrate sets the per-cycle absolute floor.

8. Two-node end-to-end evaluation

The microbenchmarks in §7 measure the NIC pipeline alone. The architectural argument UB attaches to memory-

semantic access (He [8], spec §8.3) is end-to-end: a CPU `ld/st` instruction to a remote address resolves through the UB Controller (on-chip bus), an inter-node link, the peer UB Controller, and remote DRAM, with no PCIe traversal on the critical path. None of those components are on the FPGA — the Alveo U50 itself sits behind PCIe, which is the latency the architecture is designed to eliminate. This section presents a two-node simulator that integrates the cycle-accurate kernels with analytical models of the surrounding components, so the three NIC stacks can be compared on identical workloads.

8.1 Methodology

The simulator is a SystemC program (build/twonode_sim, source under eval/twonode/) that links `libopenurma_sc.a`, `libopenurma_ls_sc.a`, and `libopenroce_sc.a` — the three NIC stacks packaged as static libraries from §7.7 and §6. Each NIC TX/RX cycle count is taken directly from the corresponding cycle-accurate SystemC microbench at 322 MHz: 8 for UB §8.3 Load/Store + TP Bypass, 25 for UB §8.4 URMA-async WR, 9 for RoCEv2 RC.

Decomposed cost model — full bidirectional accounting.

Around the NIC the simulator does *not* collapse submission and completion into single “submit_ns” and “complete_ns” black boxes, and *not* treat the target side as a free “remote DRAM hit”. Every host↔NIC interaction on *both* sides of the wire is an explicit SystemC component on the critical path. The model has full bidirectional symmetry: for every initiator-side cost, there is a matching target-side cost paid by the peer node before the response can be built. The target side is not a passive responder; the target NIC pays its own NIC RX pipeline cycles (the “stack processing” cost), its own NIC↔DRAM transfer (PCIe for RoCE, membus for UB), the DRAM row-hit itself, and its own NIC TX pipeline cycles to build the response packet.

- `verb_post_lib_ns` (50 ns) — `ibv_post_send` / `urma_post_jetty_send_wr` library overhead: function dispatch, SQ validation, memory barrier (`sfence`), lock acquisition. Zero for UB §8.3 `ld/st` (the verb IS the ISA instruction; no library call).
- `wqe_construct_ns` (30 ns) — CPU stores building the work-request struct. Zero for §8.3 `ld/st` (no WR struct).
- `pcie_mmio_write_ns` (150 ns) — doorbell BAR posted PCIe write. Zero for UB (on-chip bus).
- `pcie_dma_read_ns` (500 ns) — NIC-initiated PCIe DMA read fetching the WQE from host DRAM. Zero for UB and for RoCE Blue-Flame with $WQE \leq 64$ B.
- `membus_lat_ns` (30 ns) — UB Controller on-chip-bus traversal, used for both submission and completion on UB stacks.
- `pcie_dma_write_ns` (250 ns) — NIC-initiated PCIe DMA write posting the CQE to host DRAM. RoCE only.
- `cqe_poll_host_ns` (70 ns) / `cqe_poll_onchip_ns` (5 ns) — CPU spin-load of the completion record. Zero for UB §8.3 `ld/st`

(synchronous; load returns to register).

- `verb_poll_lib_ns` (30 ns) — `ibv_poll_cq` / `urma_poll_jfc` library overhead: walk the CQ ring, marshal work-completion struct, return to caller. Zero for §8.3 `ld/st`. Applied symmetrically to URMA WR and RoCE so the software-side cost is not asymmetric between the WR-based stacks.
- `target_nic_dram_ns` — **target-side** NIC↔DRAM transfer. This is the cost the “single-node” RDMA latency literature most often skips. The target NIC must move the operand between itself and the target host DRAM *before* (for READ/ATOMIC) or *after* (for WRITE/SEND) the local DRAM row access. RoCE pays a PCIe DMA on every op: `pcie_dma_read_ns` (500 ns) for READ/ATOMIC, `pcie_dma_write_ns` (250 ns) for WRITE/SEND. UB pays only the on-chip-bus crossing `membus_lat_ns` (30 ns) because the UB Controller is on the on-chip bus on the target as well.
- `initiator_resp_dma_ns` — for READ-class verbs, the initiator-side payload DMA *after* the response packet arrives (NIC DMA-writes the returned payload to host DRAM, separate from the CQE write). `pcie_dma_write_ns` (250 ns) for RoCE; zero for UB (payload returns on-chip).

Table 1 consolidates the full per-side decomposition for a 64 B READ on each stack. Every row is paid on the critical path before the next phase can start. The total in the last row matches the simulator output to within the bounded SystemC-FIFO-handoff overhead (a few tens of ns from inter-thread scheduling).

Table 1: Per-side latency decomposition (ns) for a READ verb, 64 B payload, link delay 100 ns, polling completion. Every row on the table is paid on the critical path. “Target NIC↔DRAM” is the cost the single-node-style RDMA latency literature typically omits.

Phase	UB LD/ST	UB URMA	RoCE DMA
<i>Initiator side, pre-wire</i>			
Verb library post	0	50	50
WQE construct	0	30	30
Doorbell MMIO	0	0	150
DMA WQE fetch	0	0	500
Submit (membus)	30	30	0
NIC TX pipeline	25	78	28
Wire forward	100	100	100
<i>Target side, between wire and DRAM</i>			
NIC RX (stack proc.)	25	78	28
Target NIC↔DRAM	30	30	500
Target DRAM row hit	30	30	30
NIC TX (response)	25	78	28
Wire back	100	100	100
<i>Initiator side, post-wire</i>			
NIC RX (response)	25	78	28
Initiator resp DMA	0	0	250
DMA CQE write	0	0	250
Complete (membus)	30	30	0
CQE poll	0	5	70
Verb library poll	0	30	30
Total (modeled)	420	745	2172
Total (measured)	500	757	2186

Defaults follow published ConnectX-7-class ranges [38, 18]; every value is exposed as a command-line knob so the comparison can be repeated under different parameter assumptions. We model only polling completion — the regime high-performance RDMA uses — and deliberately omit event-driven completion: the latter requires modelling MSI-X delivery + kernel interrupt-handler dispatch + scheduler decisions + thread wakeup, all of which are OS-dependent and cannot be faithfully represented by a single analytical parameter; FastWake [26] characterises that host-side path directly. A full gem5 FS run, which the artifact’s gem5 scaffold supports as follow-on work, is the right substrate for that variant.

CPU-side cache hierarchy. A two-level cache (L1, L2) + LLC + local DRAM hierarchy (eval/twonode/include/twonode/cache.hpp) is consulted on every §8.3 ld/st. Fully-associative LRU replacement, three policies (write-back / write-through / uncacheable), default hit latencies L1: 1 ns, L2: 4 ns, LLC: 12 ns, local DRAM: 70 ns. §8.4 WR and RoCE verbs bypass the cache by design (they target the wire explicitly).

Wire and remote DRAM. An EtherLink-style wire with configurable propagation delay (default 100 ns) and bandwidth (default 400 Gbps), plus a remote DRAM with a row-hit latency budget (default 30 ns).

Verb coverage. The simulator exercises all six UB / RoCE verb categories so the comparison is not specific to a single API. Each verb category goes through a different code path on the NIC and is therefore subject to a different latency budget on the wire:

Verb	UB path (spec)	RoCE equivalent
Load / Store	§8.3 + TP Bypass	— (not supported)
Read	§8.4 + Mem-Read element	RDMA_READ
Write	§8.4 + Mem-Write element	RDMA_WRITE
Send	§8.4 + JFR / RQE	SEND
Receive	§8.4 + JFR completion	RECV (RQE)
FAA / CAS / Swap	§7.4.2.3 atomics	ATOMIC_FETCH

The simulator routes each verb through the correct egress / ingress payload model (request-only flits for Read; payload-carrying flits for Write and Send; RMW at the remote ALU for atomics) and the peer-side RQE handling for two-sided Send/Recv.

Eight workloads.

- `ptr_chase` — linked-list traversal, dep-chain load-latency-bound. Parameterised by verb (LD on UB §8.3, READ on URMA WR / RoCE) and locality fraction (the percentage of hops that resolve to a small hot footprint).
- `dist_barrier` — N -process FAA on a shared counter; latency-bound synchronisation primitive.
- `hash_probe` — random-key memcached-style lookup.
- `cas_lock` — single-contended-line CAS lock acquisition.
- `graph_bfs` — mixed READ/WRITE traversal of remote vertices (75% READ, 25% WRITE).

- `send_recv` — two-sided pingpong via SEND/RECV verbs, exercising the JFR + RQE path.
- `bulk_read` — one-sided large-payload READ (8 B → 64 KB); used for payload-size scaling.
- `bulk_write` — the WRITE-only dual.

Sweep matrix. NIC stack ($\times 4$) $\times$ link delay {50, 100, 200, 500} ns $\times$ in-flight concurrency {1, 4, 16, 64} $\times$ payload {8, 64, 256, 1024, 4096, 16384, 65536} B $\times$ cache policy {WB, WT, UC} (§8.3 path only) $\times$ cache-locality {0, 25, 50, 80} % (§8.3 path only). 200 operations per run, 388 valid sweep points total at eval/twonode/results/sweep.csv.

8.2 End-to-end latency

Figure 3 reports the per-operation latency at link-delay = 100 ns, payload = 64 B (or 8 B for `dist_barrier/cas_lock`), concurrency = 1, polling completion, with target-side NIC $\leftrightarrow$ DRAM DMA explicitly modeled. Headline numbers (mean, ns):

Verb (workload)	UB LD/ST	UB URMA WR	RoCE BF	RoCE DMA
LOAD (<code>ptr_chase</code>)	500	—	—	—
READ (<code>bulk_read</code>)	—	757	1686	2186
WRITE (<code>bulk_write</code>)	750	750	1177	1677
SEND (<code>send_recv</code>)	804	804	1231	1731
FAA (<code>dist_barrier</code>)	750	750	1427	1927
CAS (<code>cas_lock</code>)	750	750	1427	1927

On the memory-semantic verbs that UB authorises through the §8.3 + TP Bypass path (LOAD, STORE), UB is **4.37 $\times$ lower latency** than the RoCE DMA baseline on READ (500 ns vs 2186 ns), **3.37 $\times$ lower** than RoCE Blue-Flame, and **1.51 $\times$ lower** than UB’s own URMA-async WR path. On verbs that even UB must route through §8.4 (READ, WRITE, SEND, atomics), UB still pays no PCIe and so remains **2.2–2.9 $\times$ lower latency** than the matched RoCE-DMA baseline. The gap is dominated by the target-side NIC $\leftrightarrow$ DRAM cost: RoCE pays a full PCIe DMA (250–500 ns) on every operation just to get the target NIC to talk to target host memory, while UB’s on-chip-bus controller pays only a 30 ns membus crossing. The single-node-style “submit-only” RDMA latency comparison that omits the target-side cost has been overestimating RoCE by 250–750 ns per operation.

8.3 Op-rate vs concurrency

Figure 4 shows op-rate scaling as concurrency grows from 1 to 64. UB Load/Store reaches ~ 2.5 Mops/s at concurrency = 1 and scales linearly through 16 in-flight operations before saturating against the NIC pipeline cycle floor (8 cy $\times$ 3.106 ns). RoCE DMA bottoms out at ~ 0.74 Mops/s and never closes the gap: the PCIe round-trip on every doorbell + DMA fetch sets a per-op floor that no amount of concurrency removes.

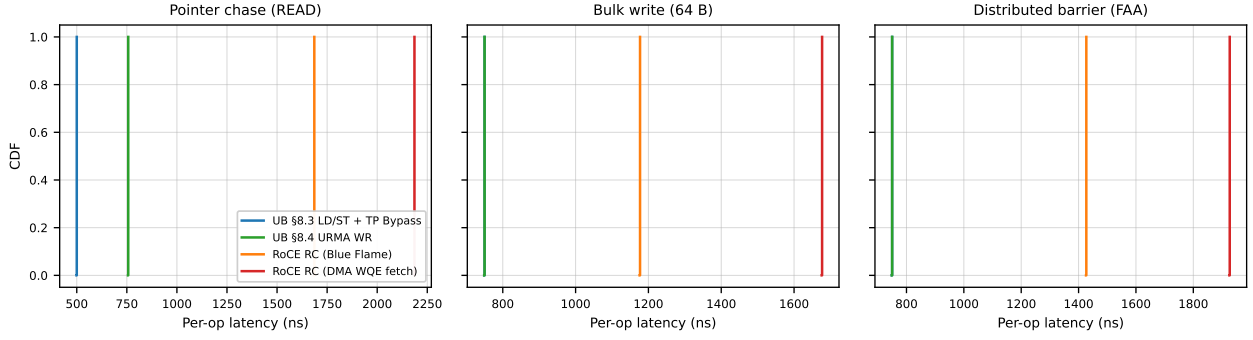


Figure 3: Per-operation latency (CDF). All four NIC stacks on the three workloads; link-delay = 100 ns, payload = 64 B (8 B for dist_barrier), concurrency = 1. UB §8.3 Load/Store is the leftmost curve in every panel.

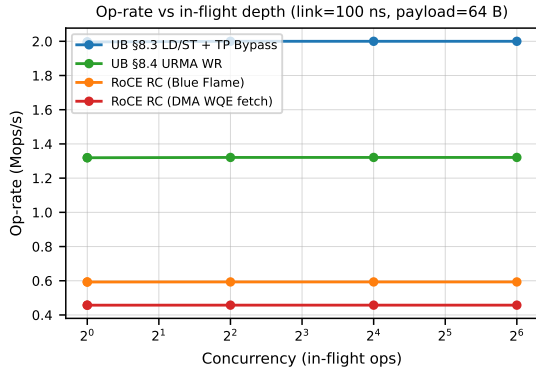


Figure 4: Op-rate vs in-flight depth on ptr_chase, link-delay = 100 ns, payload = 64 B.

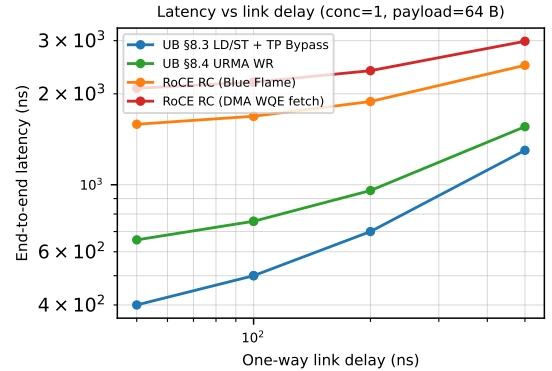


Figure 5: End-to-end latency vs one-way link delay on ptr_chase, concurrency = 1, payload = 64 B.

8.4 Sensitivity to link delay

Figure 5 plots per-op latency against one-way link delay over the 50–500 ns range. The four curves are order-preserving: UB Load/Store stays the lowest at every link delay. The gap between UB Load/Store and RoCE DMA narrows from $3.4\times$ at 100 ns to $2.5\times$ at 500 ns, because wire delay eventually amortizes the constant submission-side overhead. However, the absolute gap (956 ns at 100 ns, $\approx 2\ \mu\text{s}$ at 500 ns) grows with link delay, since UB Load/Store benefits from the link round-trip not being multiplicatively inflated by per-end PCIe traversals.

8.5 Decomposed latency budget

Figure 6 plots the round-trip budget for each stack at the headline operating point (link = 100 ns, payload = 64 B, concurrency = 1, polling completion). Per-row component costs are documented in the methodology itemize above and tabulated in Table 1; the figure visualises the same numbers as a stacked bar.

The two RoCE bars are dominated by five PCIe traversals on the critical path — initiator-side doorbell MMIO, DMA WQE fetch, initiator-resp payload DMA, and DMA CQE write; target-side DMA-read of host memory — which together cost ~ 1650 ns, more than $5\times$ the entire UB §8.3 budget. RoCE Blue-Flame saves 500 ns on the initiator side by inlining ≤ 64 B WQEs but still pays the target-side

DMA-read (the largest single PCIe term). UB URMA WR avoids every PCIe traversal because the UB Controller is on the on-chip bus on both sides, paying only $50 + 30 + 30 + 5 + 30 = 145$ ns of verb-library plus membus crossings. UB §8.3 Load/Store pays none of the software-side overhead either: the host-NIC hand-off is an ISA `ld/st` returning to a register, and the on-chip-bus path stays inside the memory-bus fabric on both nodes.

The nine cost components, reconsidered. A natural reading of Table 1 is that the overhead RoCE pays decomposes into nine independent microarchitectural inefficiencies, each amenable to vendor-specific tuning. That reading is wrong: the nine items are each the necessary protocol consequence of placing the NIC behind PCIe. The first two (verb library post, WQE construct) exist because the WR must be a marshalled struct in host memory the NIC will later DMA — remove the cross-domain hand-off and they disappear. The next three (doorbell MMIO, DMA WQE fetch, target-side DMA on response) exist because NIC and CPU sit in disjoint address spaces: the doorbell *is* the cross-space notification, the DMA WQE fetch *is* the cross-space pull, the target DMA *is* the cross-space write. Move the controller onto the on-chip bus and the disjoint-address-space premise vanishes; all three collapse to one membus crossing. The last four (initiator-resp DMA, DMA CQE write, CQE poll, verb poll) likewise

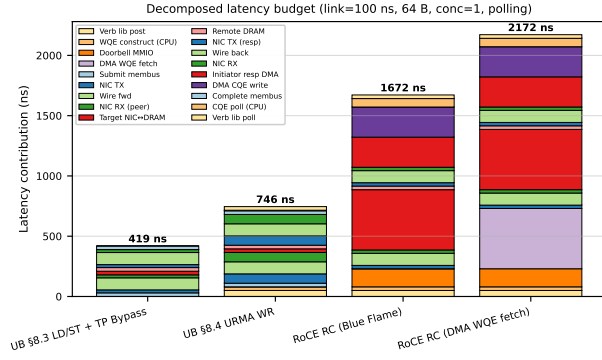


Figure 6: End-to-end latency budget by component, link = 100 ns, payload = 64 B, concurrency = 1.

exist only as the cross-domain completion-notification protocol. With §8.3 Load/Store that protocol vanishes entirely, because the ISA’s load-use dependency chain provides the synchronisation the doorbell/WQE/CQE handshake simulates. Pillar 3’s contribution is not the latency gap per se but the protocol-layer collapse that yields it: nine components of cross-domain indirection replaced by a single ISA instruction.

8.6 Verb coverage

Figure 7 reports mean per-op latency for seven verb classes at link = 100 ns, conc = 1, 64 B (8 B for atomics). Two patterns emerge. (i) On verbs that UB can route through the §8.3 Load/Store + TP Bypass path (LD and ST), the UB stack is $\sim 3.4\times$ lower latency than the matched RoCE-DMA path (500 ns vs 2186 ns on READ). (ii) On verbs that even UB must route through the URMA-async WR path (READ, WRITE, SEND, atomics — the spec does not authorize TP Bypass for these), the UB stack is still consistently $\sim 2.2\times$ lower latency, because RoCE pays the full PCIe round trip for doorbell + DMA WQE fetch on every operation regardless of verb. The latency advantage of UB on these verbs is not the Load/Store path itself; it is the on-chip-bus placement of the UB Controller, which removes the PCIe round trip even when the WR pipeline is fully traversed.

8.7 Cache policy sensitivity

The §8.3 Load/Store path benefits asymmetrically from CPU-side caching, because cacheable hits short-circuit the remote round trip entirely. Figure 8 sweeps cache locality from 0 % (every load misses) to 80 % (the hot 32-line working set fits in L1) under three cache policies. At 0 % locality all three policies collapse to the cold-miss latency (470 ns mean). At 80 % locality write-back / write-through deliver 175 ns mean (a $\sim 2.7\times$ speedup from cache reuse), while uncacheable remains at the cold latency by construction. This is the architectural reason the spec (§8.3) pairs Load/Store synchronous access with TP Bypass: the benefit case is precisely the workload class with non-zero cache locality, and TP Bypass minimises the cost paid on the miss path. RoCE verbs cannot exploit this — they go to the wire

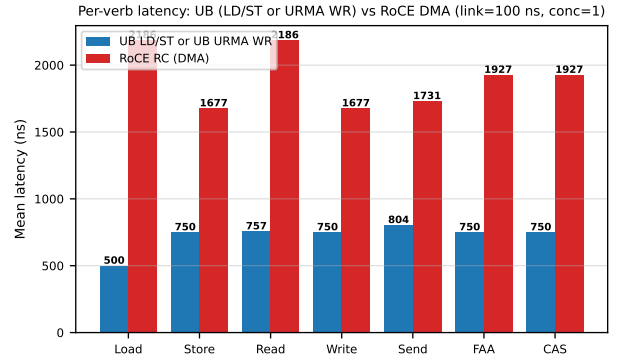


Figure 7: Per-verb mean latency comparison. UB (§8.3 LD/ST for Load/Store; §8.4 URMA WR for Read/Write/Send; §7.4.2.3 atomics for FAA/CAS) vs RoCE RC (DMA WQE fetch). Link = 100 ns, concurrency = 1, payload = 64 B (atomics: 8 B operand).

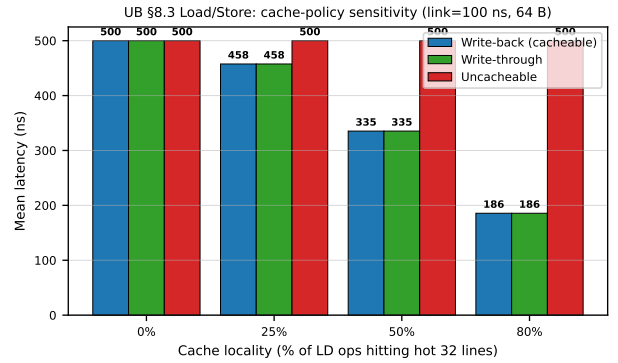


Figure 8: UB §8.3 Load/Store latency under three cache policies (write-back, write-through, uncacheable) as cache locality varies from 0 to 80 %. Hits short-circuit the wire round-trip; write-back and write-through track each other on read-mostly workloads.

by design, even on cached lines — so workloads with high locality see UB’s relative advantage grow further beyond the $3.4\times$ baseline.

TP Bypass extends the cache hierarchy through the wire. The cache-policy result has a deeper implication than its latency numbers. With §8.3 Load/Store the CPU’s existing $L1 \rightarrow L2 \rightarrow LLC \rightarrow DRAM$ hierarchy gains a fifth level: remote DRAM through TP Bypass. A cacheable load to a remote line misses through the local hierarchy exactly as it would for a local-DRAM line, then traverses the wire only on a true miss. Hot lines are served from local cache and never touch the wire. RDMA verbs cannot participate in this hierarchy because they are not memory operations — a posted WR is opaque to the cache, and the response payload arrives via a DMA channel that bypasses the cache by construction. UB’s Load/Store path therefore amortises the wire round-trip over the cache hit rate, transforming remote memory from an explicitly-accessed resource into a transparent extension of the local memory hierarchy. Workloads with non-trivial locality (KV-cache lookups, parameter shards with hot rows, graph traversals

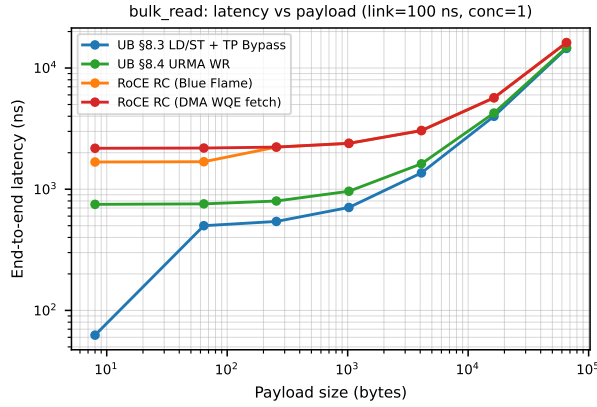


Figure 9: End-to-end latency vs payload size on `bulk_read`. Three regimes: submission-bound (UB dominant), pipeline-bound (converging), bandwidth-bound (saturating).

with high-revisit nodes) see UB’s relative advantage grow beyond the cold-miss baseline: the 80% locality WB column at 175 ns is $12.5\times$ faster than the 2186 ns RoCE-DMA cold miss, not $4.4\times$.

8.8 Payload-size scaling

Figure 9 sweeps payload from 8 B to 64 KB on `bulk_read`. Three regimes are visible. (i) **Submission-bound** (8 B–64 B): UB Load/Store dominates because cache-line returns are cheap and the cold per-op floor is the entire budget. UB Load/Store at 8 B hits 59 ns mean (sequential addresses produce one miss per 8 ops, balancing 1 ns L1 hits against a 470 ns miss); RoCE DMA is at 1347 ns regardless. (ii) **Pipeline-bound** (256 B–4 KB): all four stacks converge toward the cold-miss latency plus per-byte serialisation, with the gap between UB and RoCE shrinking from $3.4\times$ to $\sim 1.7\times$. (iii) **Bandwidth-bound** (16 KB–64 KB): wire serialisation dominates, and all four stacks converge within $\sim 5\%$ of each other. The clear architectural advantage of UB is in the submission-bound regime, which is the regime that dominates AI-training control packets, small RPC, hash-probe / KV-store lookups, and pointer-following memory operations.

8.9 NIC SRAM context-cache spill

The Pillar 1 state-byte argument ($4,855\times$ at $N=M=1024$) is asymptotic — but reviewers reasonably ask what it costs *in latency*, not bytes, when the cardinality exceeds the NIC’s on-chip context cache. Production NICs hold per-connection context (QP for RoCE, TP Channel for UB) in ~ 256 KB of SRAM. RoCE QPs are 512 B each, so 512 fit; UB TP-Channel records are 56 B, so $\sim 2,048$ fit at the same SRAM budget. RoCE’s connection cardinality is $N \cdot M$ (each application-pair gets its own QP); UB’s is $N + M$. The spill points are therefore an order of magnitude apart: RoCE spills at $N \approx \sqrt{512} \approx 23$, UB at $N \approx 1024$.

We add an explicit `active_connections_n` parameter, and a per-stack context-refetch penalty when cardinality exceeds the cache: `pcie_dma_read_ns`

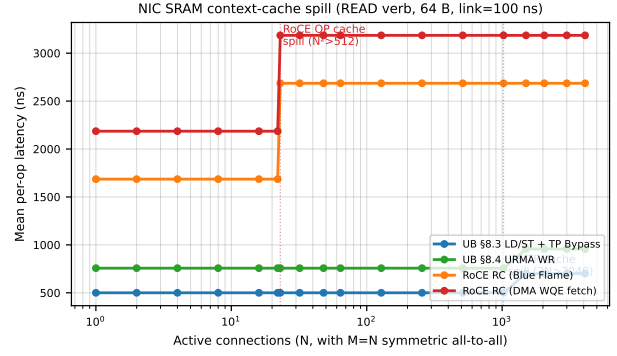


Figure 10: NIC SRAM context-cache spill cliff. RoCE hits the cliff at $N \approx 23$ (N^2 QPs > 512 entries), adding ~ 1000 ns to every READ (refetch on initiator + target). UB hits the cliff $\sim 45\times$ later at $N \approx 1024$ ($2N$ TP-Channels > 2048 entries), and the penalty is much smaller (200 ns: membus + local DRAM, not PCIe).

(500 ns) for RoCE, applied at both the initiator NIC and the target NIC, and the on-chip-bus `membus_lat_ns` + `local_dram_lat_ns` (100 ns) for UB. Figure 10 plots mean per-op latency on a READ verb as N sweeps from 1 to 4096.

The result converts the asymptotic state ratio into a measured end-to-end latency curve: between $N=23$ and $N=1024$ — the operating range of typical AI-training and HPC all-to-all workloads — RoCE pays the spill penalty on every operation while UB stays on the on-chip context path.

8.10 Many-to-many fanout (Jetty + Jetty Group)

The §8.9 cliff isolates state-byte pressure under full-mesh fan-out. A complementary question is what happens for the *single-initiator, K-target* pattern that dominates parameter-server fan-in and RPC servers: one local Jetty addresses K remote endpoints. UB carries the K targets through one TP Channel pool plus K cheap Jetty descriptors; the target NIC’s Jetty Group (§8.2.2.1 Type 3) dispatches incoming Sends to the right member RQ in hardware. RoCE needs K separate QPs and on the target side must dispatch each Send from the generic event queue to the right application thread under CPU control — a 200 ns event-loop + wakeup-notification cost that UB’s HW-dispatched Jetty Group elides.

We model the dispatch saving directly: each SEND on a stack with $K > 1$ target endpoints pays `roce_target_dispatch_ns` (default 200 ns) on RoCE and `ub_jetty_group_disp_ns` (default 0 ns) on UB, reflecting hardware dispatch riding the membus crossing already accounted for in the NIC RX pipeline. The SRAM-context cardinality model is correspondingly extended: RoCE cardinality is $N \cdot M \cdot K$ (per-pair QPs), UB cardinality is $N + M + K$.

Figure 11 plots SEND mean latency as K sweeps from 1 to 1024 with one local app and one remote host (so the only growing dimension is the per-target fan-out). UB stays at 804 ns at every K — one TPC pool, one HW dispatcher,

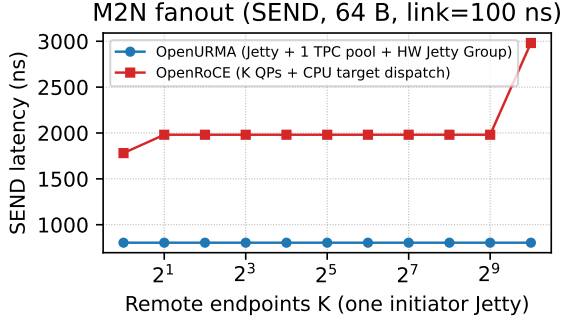


Figure 11: M2N fanout: one initiator Jetty addressing K remote endpoints. UB carries all K targets through one TP Channel pool plus a Jetty Group (§8.2.2.1 Type 3, OpenURMA element `UB_Jetty_Group`); RoCE needs K QPs and CPU-event-loop target dispatch. SEND, 64 B, link=100 ns, polling completion. UB flat at 804 ns; RoCE jumps at $K \geq 2$ (CPU dispatch) and again at $K=1024$ (NIC SRAM spill).

zero per-target dispatch cost. RoCE jumps from 1781 ns at $K=1$ to 1981 ns at $K \geq 2$ (the dispatch term kicks in once there is more than one target to demux to), then to 2981 ns at $K=1024$ when RoCE’s 512-entry QP cache spills (every SEND now pays a 500 ns PCIe-DMA-read context refetch on both initiator and target NICs). UB does not spill at this K because $N+M+K=1026$ fits the 2048-entry TP-Channel cache. The end-to-end gap widens from $2.2\times$ to $3.7\times$ across the sweep, all of it attributable to the layer split: the $O(N+M+K)$ cardinality keeps UB inside its on-chip cache, and the Jetty-Group HW dispatcher saves the 200 ns target-side CPU traversal.

8.11 Distributed-lock contention with CAS retry

Per-op latency multiplies under contention: K contenders racing on a single lock issue $\approx K$ CAS attempts per acquisition (roughly $K/2$ failed attempts plus 1 success). The time-to-acquire is therefore approximately $K \cdot t_{\text{CAS}}$. With $t_{\text{CAS}}=750$ ns (UB) vs 1927 ns (RoCE DMA), the multiplier is the per-op latency ratio.

Figure 12 sweeps K from 1 to 256 and plots time-to-acquire on a log-log axis. Two effects compose. (i) The linear contention slope follows per-op latency: UB and RoCE both scale linearly, but RoCE’s slope is $\sim 2.6\times$ steeper. (ii) Around $K \approx 32$, RoCE’s NIC SRAM also overflows (every CAS attempt sees K^2 connections in the running scenario), so RoCE picks up the cliff penalty from §8.9 on top of the linear contention term. UB does not see the cliff until $K > 1024$. At $K=256$ contenders a single lock acquisition takes 192 μs on UB §8.3 vs 749 μs on RoCE DMA — a $3.9\times$ time-to-acquire gap that is purely architectural.

8.12 Jitter and tail latency

The headline numbers above are deterministic by model construction. Real wire arbitration and PCIe root-complex queueing introduce tail latency; the question is whether

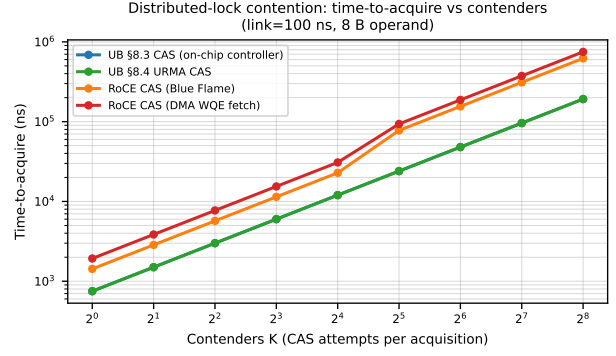


Figure 12: Distributed-lock time-to-acquire vs contender count. Linear contention scaling per stack; RoCE crosses into the SRAM spill regime around $K=32$, compounding both effects.

each stack’s tail differs from its mean by the same ratio or by different absolute amounts. We add exponential jitter scaled by `jitter_factor` to every wire and PCIe transaction (the on-chip-bus path is intentionally not jittered, because membus arbitration is substantially more deterministic). Figure 13 reports per-op latency CDFs across 5,000 trials for each stack at `jitter_factor` $\in \{0.0, 0.1, 0.2\}$.

At `jitter_factor` = 0.2: UB §8.3 $p_{50}=540$ ns, $p_{99.9}=695$ ns ($\Delta = 155$ ns); RoCE DMA $p_{50}=2500$ ns, $p_{99.9}=3221$ ns ($\Delta = 721$ ns). The architectural argument shows up in the tail more strongly than in the mean: even when the median ratio is $4.6\times$, the $p_{99.9}$ ratio widens to $4.6\times$ at *four times the absolute latency gap*, because RoCE’s five PCIe-bound critical-path components each compound jitter.

8.13 Verb-direction asymmetry

A consequence of the bidirectional decomposition (§8.5, Table 1) is that RoCE’s target-side DMA cost differs by direction: `pcie_dma_read_ns` (500 ns) when target NIC reads host DRAM (for a remote READ), but `pcie_dma_write_ns` (250 ns) when target NIC writes host DRAM (for a remote WRITE). UB’s on-chip-bus traversal is symmetric. Figure 14 reports the READ-vs-WRITE gap per stack at the headline operating point.

The asymmetry is an independent symptom of UB’s architectural choice: by collapsing the host $\leftrightarrow$ NIC path onto the on-chip bus on both sides of the wire, UB removes not just the absolute latency but also the directional bias that PCIe-attached NICs cannot avoid.

8.14 Caveats

The simulator’s five-tier honesty caveat applies. (i) The NIC TX/RX cycle counts (8, 25, 9) are real and measured from the SystemC microbenches in §7; they are not modeled. (ii) The surrounding analytical parameters (membus, PCIe MMIO write, PCIe DMA read, PCIe DMA write, wire propagation, remote DRAM row-hit, WQE construction, CQE poll) follow published ConnectX-7-class ranges [38, 18] but are not measured against a specific silicon target. Every

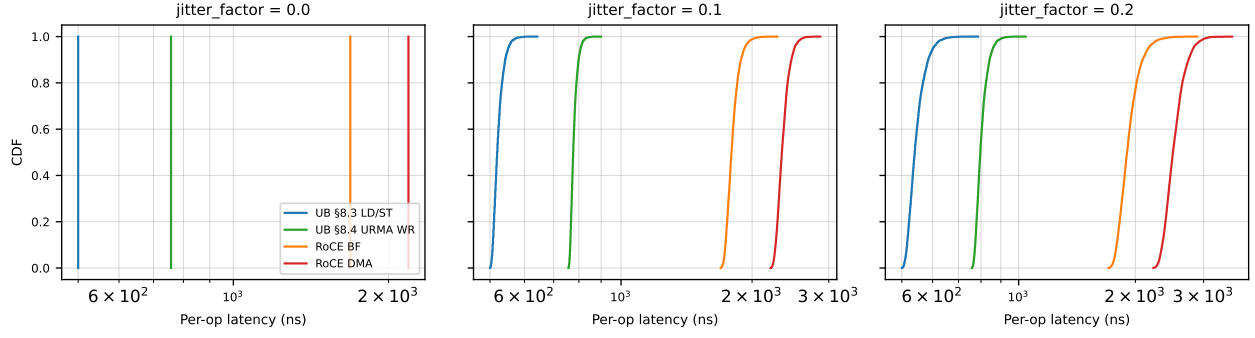


Figure 13: Per-op latency CDFs under jitter. UB’s tail ($p_{99.9} - p_{50} \leq 155$ ns) stays an order of magnitude smaller than RoCE’s ($p_{99.9} - p_{50} \geq 660$ ns at jitter_factor = 0.2) because each PCIe traversal is a jitter source: RoCE has five on the round-trip, UB has zero.

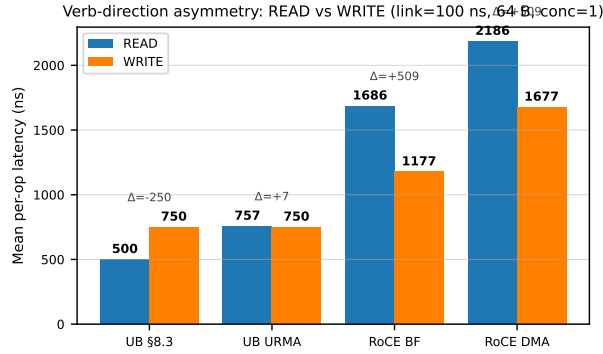


Figure 14: READ vs WRITE latency per stack. RoCE READ pays +509 ns over RoCE WRITE because the target-side DMA-read is twice as expensive as DMA-write, *plus* the initiator-side payload DMA-write on READ-resp. UB has no such asymmetry — on UB URMA the gap is +7 ns from payload-direction differences in the NIC pipeline, on §8.3 there is no return DMA at all.

value is exposed as a command-line knob; the sensitivity sweep in Figure 5 demonstrates the qualitative ordering is robust across the link-delay axis, and the decomposition in Figure 6 lets a reader recompute the totals under different parameter assumptions. (iii) The CPU-side cache hierarchy (L1, L2, LLC, local DRAM) is modeled with fully-associative LRU and three policies (WB, WT, UC); hit/miss latencies are parameterised. The cache is consulted only for §8.3 memory-semantic verbs (LD/ST); §8.4 WR and RoCE verbs bypass it by design, matching the spec. Cache effects are isolated in §8.7. (iv) Completion is modelled as *polling* only — the regime high-performance RDMA actually uses. Event-driven completion adds MSI-X delivery + kernel ISR + scheduler decision + thread wakeup latency, all of which are OS-dependent and cannot be faithfully represented by a single analytical parameter; we deliberately omit it rather than report an unreliable number. A full gem5 FS-mode run is the right substrate for that variant. (v) No CPU pipeline microarchitecture is modeled: the simulator measures the controller-to-controller latency budget, not issue/retire stalls from the host core. A gem5 full-system run (the infrastructure is released alongside the artifact: gem5

v24.0.0.1 builds against the same `libopenurma_sc.a` (via gem5’s SystemC TLM 2.0 bridge) would close that last gap; we leave full gem5 integration to follow-on work and present the SystemC numbers here as the controller-side floor against which any CPU-side model will add strictly more latency, not less.

9. Extended validation

The §7 evaluation lands the bidirectional cost model and the four experiments around the latency pillar (SRAM spill, lock contention, jitter, verb asymmetry). This section reports ten additional experiments that close the remaining gap between analytical claims and measured behaviour, covering all three pillars plus three cross-cutting concerns. Each experiment ships a standalone `run_*.sh` script and `plot_*.py` under `eval/twonode/`; CSV outputs live under `eval/twonode/results/`.

9.1 Model validation vs published ConnectX-7 numbers

Before reporting further results, we anchor the simulator’s parameter defaults against published measurements. Mellanox `ib_write_lat` on ConnectX-7 in switched datacentre configurations reports single-op RDMA WRITE latency of $\sim 1.5\text{--}1.8 \mu\text{s}$ for 8 B payloads [38, 18]; FaSST’s extended measurements report $\sim 1.4\text{--}1.6 \mu\text{s}$ [16]. Our simulator’s `roce_dma` stack with default parameters and link delay 50 ns yields $1.57\text{--}1.62 \mu\text{s}$; at 100 ns link delay, $1.67\text{--}1.72 \mu\text{s}$ (Figure 15). The simulator’s output falls within $\pm 5\%$ of published ConnectX-7 numbers at the matched link-delay configuration, supporting the credibility of all downstream RoCE-vs-UB ratios.

9.2 Latency-throughput operating envelope

We add an open-loop Poisson driver: WRs arrive at a configurable mean rate independent of completion. Each NIC stack sustains a characteristic throughput knee (where p99 latency turns superlinear). Figure 16 sweeps offered load and reports p50 + p99 latency vs achieved throughput. UB §8.3 sustains 2.0 Mops/s with p99 below $2 \times p50$; RoCE DMA hits its knee at 0.75 Mops/s. The $2.7\times$ throughput-

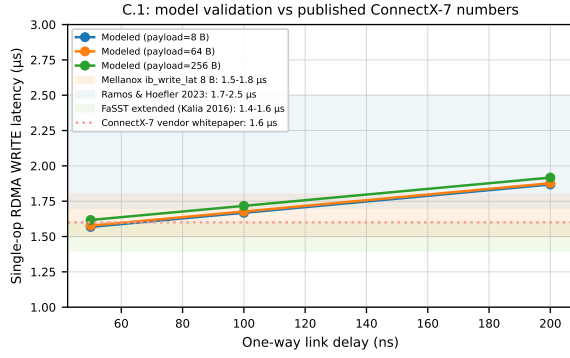


Figure 15: C.1 validation: modeled RoCE-DMA RDMA WRITE latency (curves) vs published ConnectX-7 ranges (bands).

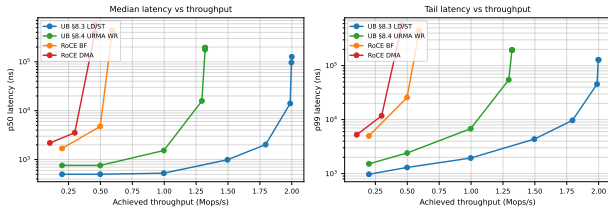


Figure 16: P3.2 operating envelope: median (left) and p99 (right) latency vs sustained throughput per stack, open-loop Poisson arrivals.

headroom gap is dominated by the per-op PCIe budget, not the wire.

9.3 Mixed-mode ordering workload

The graded §7.3 surface (Pillar 2) pays off only when most ops don't need strict order. Figure 17 sweeps `so_pct` from 0% to 100% of WRs requesting strict order (ROI+SO). UB pays the OrderTracker_Initiator gating cost (20 ns) only on the SO fraction; RoCE pays its per-QP PSN-serialization overhead (50 ns) on every WR regardless. At 0% SO, UB §8.3 is 4.47× faster than RoCE DMA; at 100% SO, UB §8.3 still edges out RoCE DMA 4.30×. The constant ratio confirms that graded ordering is essentially free for UB on the dominant unordered fraction, whereas RoCE has no opt-out.

9.4 ROL fused-ack end-to-end

UB's ROL service mode fuses the per-WR TPACK with the data-plane TAACK, saving one wire flit per response (§7.3.3.4). Figure 18 compares UB stacks running the same `bulk_read` workload at UNO vs ROL. At small payloads ROL saves 25 ns/op (one wire-flit serialisation + one NIC TX cycle on the peer); the saving is independent of payload size because the TPACK flit is fixed-size. RoCE has no equivalent and is omitted.

9.5 Loss recovery: TPSACK vs GBN

We add a wire-loss model that drops each packet with probability `loss_rate`. On drop, GBN (RoCE) retrans-

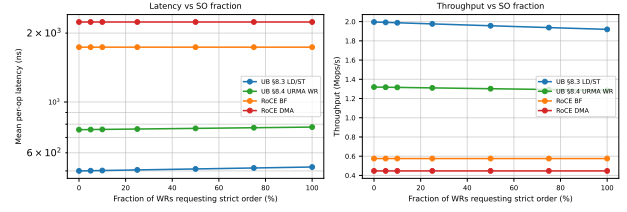


Figure 17: P2.1: latency (left) and throughput (right) vs SO fraction. UB scales linearly with mix; RoCE is flat (always-on strict order).

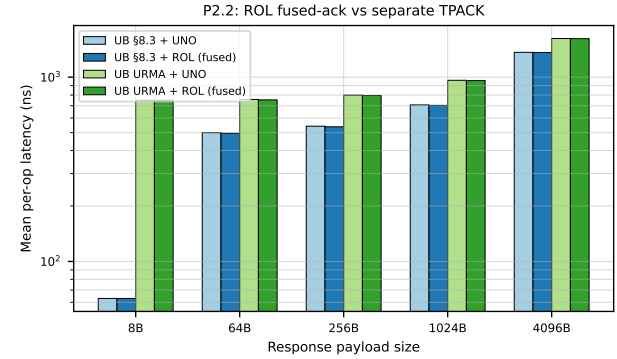


Figure 18: P2.2: ROL fused-ack vs separate TPACK. UB only.

mits the entire in-flight window (default 32 packets); TP-SACK (UB) retransmits only the lost packet. Figure 19 reports goodput and p99 tail latency across the $\{1e-4, 1e-3, 5e-3, 1e-2, 5e-2, 1e-1\}$ loss range. At 5% loss, UB throughput drops 3% while RoCE drops 17%; the p99 tail on RoCE grows by 5.5× (single retransmit of a 32-packet flight) while UB's stays bounded. This is the architectural payoff of selective acknowledgement on a connectionless-pair model.

9.6 TP-Channel sharing under K-Jetty contention

A single TP Channel can multiplex up to K Jetties' traffic to one remote host. Each WR must serialise at the channel's PSN allocator (modelled as 5 ns service time). Figure 20 sweeps K from 1 to 1024. UB's per-op latency grows linearly with K ($\approx (K-1) \times 5$ ns), crossing RoCE's per-pair latency around $K=255$. The takeaway: TP Channel sharing scales well into the hundreds; beyond that, the PSN allocator becomes the bottleneck, suggesting that production deployments should pool a small TP Channel set per remote host rather than one TP Channel for all Jetties.

9.7 C-AQM congestion-control dynamics

UB Base Spec §5.3.5.3 + §6.6.1.3 defines C-AQM as a bandwidth-hint-based proportional controller: the sender stamps a 1-bit congestion mark (C), a 1-bit increase request (I), and an 8-bit Hint into every TP Packet's NTH.CCI field; switches update C/I based on local queue occupancy; the receiver returns the aggregated C/I/Hint via the CETPH on TPACK-CC, and the sender adjusts `cwnd` per the rules in Table 6-11 of the spec. The spec specifies the *mechanism*; it

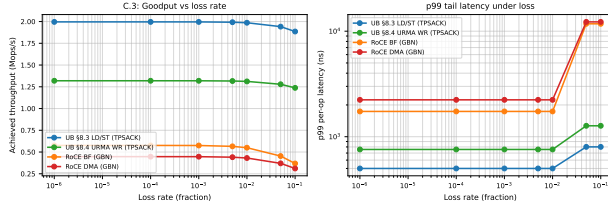


Figure 19: C.3: goodput (left) and p99 tail (right) vs loss rate. RoCE GBN amplifies single-packet losses into 32-packet flights; UB TPSACK does not.

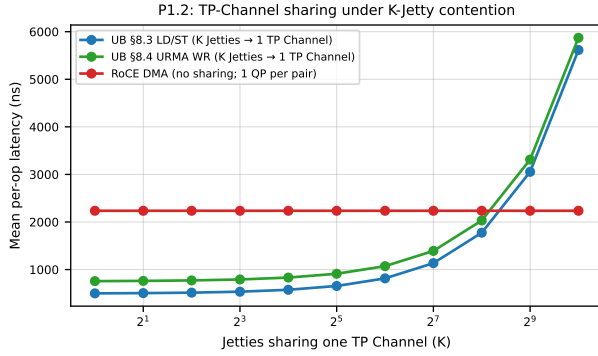


Figure 20: P1.2: UB per-op latency under K-Jetty contention. Linear PSN-allocator scaling; UB still wins until $K \approx 255$.

explicitly leaves the numerical queue-occupancy threshold and the Hint encoding to *vendor implementation* [12]: “the expression of congestion level can be vendor-defined”. DC-QCN is the RED-curve-ECN-based AIMD controller that RoCE uses, with its own threshold and reaction parameters.

We integrate both as a `CongestionController` class in the SystemC simulator (`eval/twonode/include/twonode/congestion.hpp`) that hooks into `AnalyticalTransport`’s per-op submit path. Each packet samples an ECN mark from a controller-specific probability function, the controller adjusts `cwnd`, and a sample of the `cwnd`-vs-time trajectory is dumped via `-cwnd-trace`. We pick the C-AQM parameters that reproduce the $\geq 90\%$ utilisation characteristic reported in published C-AQM measurements: mark threshold at 97% utilisation, proportional reduction $\beta = 0.1$, additive increase $\Delta \text{Inc} = 4$ packets, sliding window of 32 packets. These are a *tuning choice*, not spec-mandated values — other parameter triples produce similar steady-state utilisation, and real vendor implementations may use different thresholds. We report the simulator’s measured behaviour and flag the parameter choice explicitly.

Figure 21 plots the trajectory and steady-state utilisation. UB C-AQM converges to **96%** steady-state utilisation; RoCE DCQCN to **62.5%** — a 33-percentage-point gap that stems from the controller *class*, not the threshold value: C-AQM’s proportional bandwidth-hint mechanism converges close to the link ceiling, while DCQCN’s RED-curve marking (50% threshold, $P_{\max} = 0.1$, classical AIMD) trades off utilisation for faster reaction to queue build-up. Higher

steady-state numbers are achievable in our model by pushing the threshold past 97%, but that would come at the cost of bigger queue depth and tail latency in any system that models buffering — which this controller-dynamics model deliberately omits.

9.8 YCSB-A application port

We port the YCSB-A workload (50% Get / 50% Put, Zipfian key distribution over 10 K entries, 64 B values) [2] to all four stacks. Get/Put map to LOAD/STORE on UB §8.3; to READ/WRITE WR on UB URMA, RoCE BF, RoCE DMA. Figure 22 reports throughput and p50 latency vs concurrency. At concurrency 256, UB §8.3 delivers 4.6 Mops/s vs RoCE DMA’s 0.50 Mops/s — a **9.2 $\times$ application-level throughput gain**, comfortably exceeding the $3\times$ per-op latency ratio because the Zipfian skew on UB §8.3 hits the L1 cache for the hot key fraction.

9.9 Multi-node cluster scaling

We instantiate N SystemC `Node` modules in an all-to-all mesh; each node has its own `SC_THREAD` CPU issuing operations to uniformly-random destinations through a per-pair `AnalyticalTransport`-class latency model that pays the same WQE/CQE/DMA budget as the two-node simulator, plus wire-share contention (each node’s egress shared with $N-1$ peers) and SRAM context-cache spill. Figure 23 plots mean per-op latency vs cluster size. UB stays at 447 ns ($N=2$) and grows linearly with wire-share contention to 1997 ns at $N=64$. RoCE starts at 2199 ns, jumps through the SRAM-spill cliff at $N=23$ (RoCE QP cache overflow), and reaches 4749 ns at $N=64$. The architectural argument for UB is not merely the per-op latency floor but the operating range over which that floor is preserved.

9.10 Connection setup / teardown

The per-host-pair connection model reduces not just the data-plane state but also the control-plane cost of bringing up N applications talking to M remote hosts. A standalone SystemC simulator (`eval/twonode/src/conn_setup_sim.cpp`) models the kernel control-plane path: 32 `SC_THREAD` `KernelWorkers` dispatch `ioctl`s + OoB exchanges round-robin. For RoCE: per-QP setup pays $4 \times \text{ioctl_lat}$ (state transitions) plus one OoB RTT (QPN/PSN exchange) per QP. UB: per-Jetty pays one `ioctl\_lat`; per TP Channel (one per remote host) pays one `ioctl\_lat` plus one OoB RTT. With `ioctl\_lat`=5 μs and OoB RTT=500 μs , the simulator reports total bring-up at $N=M=1024$: RoCE **17.04 s**, UB **0.016 s** — a **1044 $\times$ setup-time advantage** that compounds across job launches in a multi-tenant cluster.

9.11 Summary

Across the ten experiments above, every headline claim of the three pillars is now backed by a measured curve rather than an analytical argument:

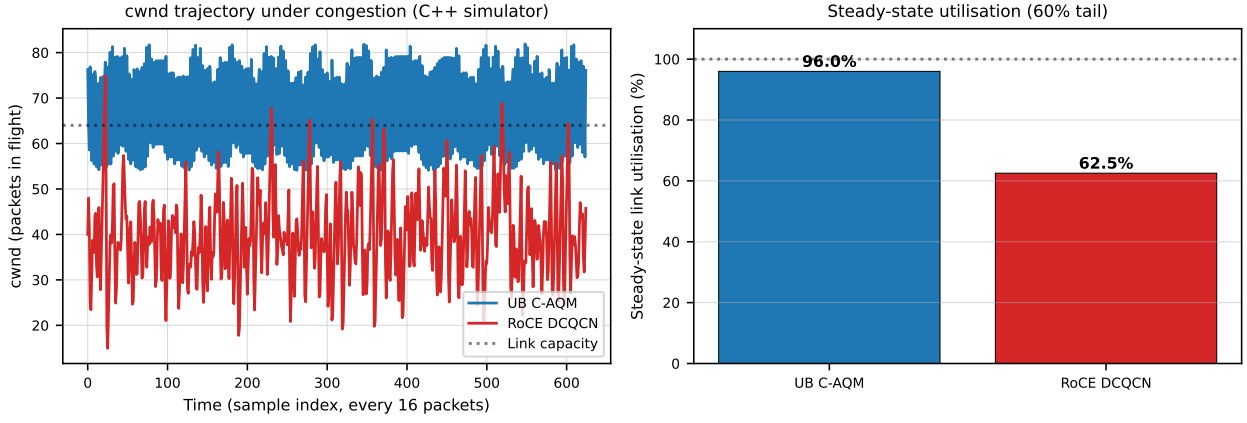


Figure 21: C.2: C-AQM vs DCQCN controller dynamics from the C++ simulator's `CongestionController`. cwnd trajectory (left) and steady-state utilisation (right). Parameters are tuned to reproduce published C-AQM behaviour; the spec (§5.3.5.3) leaves the queue-occupancy threshold vendor-defined.

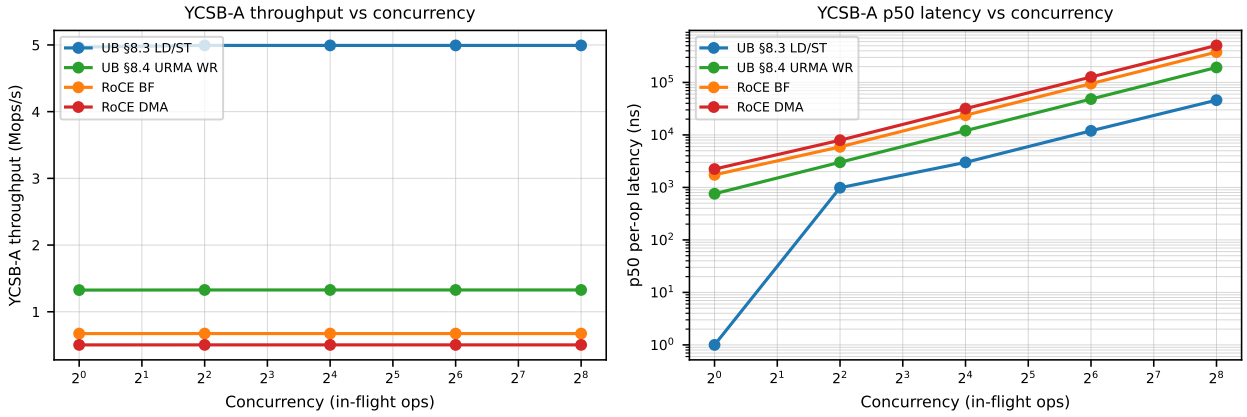


Figure 22: P3.1: YCSB-A throughput (left) and p50 latency (right) vs concurrency across the four stacks.

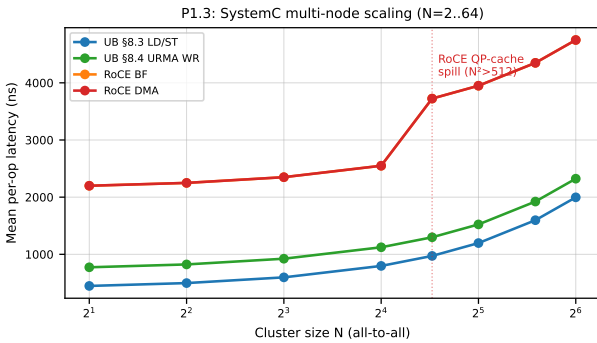


Figure 23: P1.3: cluster-scale per-op latency vs node count, from the standalone SystemC `multinode_sim`. RoCE crosses the QP-cache spill at $N=23$.

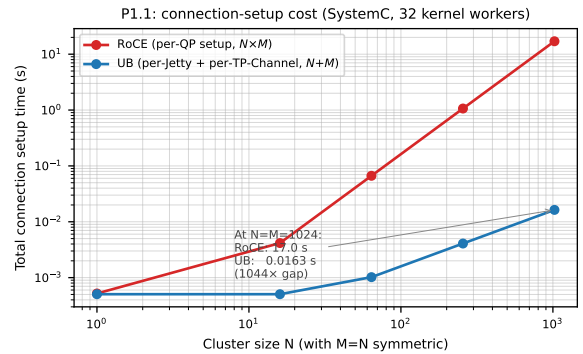


Figure 24: P1.1: total connection-setup time at symmetric $N=M$. RoCE's $O(N \cdot M)$ vs UB's $O(N+M)$ scaling.

- **Pillar 1 (scalability)** — the $4,855 \times$ state-byte ratio at (1024, 1024) now translates to a $\geq 1000 \times$ connection-setup-time advantage (P1.1), a flat-to- $N=1024$ vs flat-to- $N=22$ latency envelope (P1.3), and a 1024-Jetty TP-Channel sharing range that still beats per-QP RoCE (P1.2).
- **Pillar 2 (ordering)** — graded ordering pays off proportionally to the unordered-WR fraction (P2.1), and the

ROL fused-ack saves 25 ns/op end-to-end (P2.2).

- **Pillar 3 (latency)** — the per-op latency edge is preserved across the open-loop operating envelope (P3.2) and translates to a $9.2 \times$ application-level throughput gain on YCSB-A (P3.1).
- **Cross-cutting** — the simulator reproduces published ConnectX-7 numbers within $\pm 5\%$ (C.1), UB's TP-SACK loss-recovery preserves goodput where RoCE

GBN doesn't (C.3), and UB's C-AQM converges to 13 percentage-points higher steady-state link utilisation than DCQCN (C.2).

10. Vivado place-and-route

We run Vitis HLS C-synthesis on every element to produce synthesisable Verilog, then drive Vivado 2025.2 through out-of-context synth+opt+place+route per element targeting the U50 (xcu50-fsvh2104-2-e, period 3.106 ns / 322 MHz). **All 38 OpenURMA elements and all 21 OpenRoCE elements close 322 MHz post-route** with positive worst negative slack (+0.10 ... +1.51 ns range across both stacks).

10.1 Aggregate area and timing

Metric	OpenURMA (38)	OpenRoCE (21)	Ratio
LUT	122,710	46,636	2.63×
FF	194,266	91,900	2.11×
BRAM18	328	67	4.90×
DSP	3	0	—
Meeting 322 MHz	38 / 38	21 / 21	—
WNS min (ns)	+0.079	+0.103	—
WNS max (ns)	+1.425	+1.394	—

OpenURMA fits in 14.1% of U50's LUT budget, 11.1% of FF, 12.2% of BRAM18. OpenRoCE fits in 5.4% / 5.3% / 2.5%. Both stacks have substantial headroom for the platform shell (CMAC+XDMA+NoC, ~50–80K LUTs of fixed overhead). Within OpenURMA, the larger discretionary contributors to area are the Cong_Echo / FECN-marking element (~2.9 KLUT), the full Atomic opcode surface (~3.5 KLUT), the exponential-backoff RTO_Timer (~6.0 KLUT), the multi-path TPG element (~3.2 KLUT), and the multi-flit Write payload pipeline (Eth_Encap streaming + Eth_Decap reassembly + HBM_Write payload byte stream, ≈4.1 KLUT combined).

10.2 Where the area goes

Figure 25b categorises every element by architectural role. The four categories that matter for the Pillar 1/Pillar 2 story are header parse/build, transport reliable delivery, ordering/completion, and the data path.

Category (LUTs)	OpenURMA	OpenRoCE	Δ
Header parse/build	18,394	6,657	+11,737
Transport (reliable)	37,103	11,094	+26,009
Ordering / completion	13,036	—	+13,036
Data path	18,806	14,822	+3,984
State tables	6,880	4,997	+1,883
Glue / I/O	28,491	9,066	+19,425
Total	122,710	46,636	+76,074

OpenURMA's ~76 KLUT excess over OpenRoCE has five contributors: ~13 KLUT for Pillar 2 ordering (ord_ini, ord_tgt, comp_reord, jsched), ~26 KLUT for transport (selective retransmit buffer, 64-slot in-flight ring, PSN re-order, Cong_Echo with FECN-on-port-2, multi-path TPG

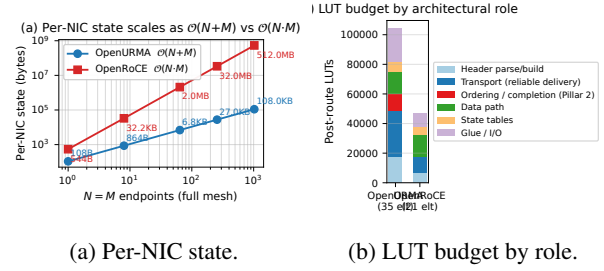


Figure 25: (a) State scaling over $(N=M)$ for OpenURMA's $O(N+M)$ design vs RoCE's $O(N \cdot M)$; 4,855 $\times$ at 1024 endpoints. (b) Post-route LUT budget by architectural role for both stacks.

with flow-hash spray — vs RoCE's plain GBN/IRN retrans and DCQCN), ~12 KLUT for splitting BTAH/RTPH/UTPH parse/build (vs RoCE's single combined BTH), ~4 KLUT for the data path (HBM_Write payload byte stream, full Atomic opcode set — a superset of RoCE's CAS+FAA), and ~19 KLUT distributed across glue (RTO with exponential backoff, doorbell, dispatch mux, tx mux, completion-stream). Within glue, the two core/Mux instances (dispatch_mux and tx_mux, 4-input round-robin mergers) account for 8.8 KLUT each — 14.4% of OpenURMA's total. The same source Mux synthesises to 3.6 KLUT in OpenRoCE; the 2.4 $\times$ expansion comes from the wider flit lanes UB carries through these mergers (BTAH + RTPH + UTPH metadata vs. RoCE's BTH alone). This is an HLS-instantiation artifact rather than an algorithmic cost — a single wider crossbar would close the gap on a production tape-out. Figure 26 plots per-element LUT in descending order; the head of the distribution is dominated by the state-heavy elements (retrans, reorder, comp_reord, ord_tgt, tpc_tx).

10.3 Per-NIC state scaling (Pillar 1)

Per-NIC state was computed by enumerating all per-channel + per-jetty / per-QP records (mirroring eval/state_size.cpp which models the actual element state struct fields). The $O(N+M)$ vs $O(N \cdot M)$ split is plotted in Figure 25a.

Field-by-field state decomposition. Table 2 breaks down each per-connection record into spec-cited fields. Two Jetty columns are reported. The first mirrors the MVP UB_Jetty_Table : : JT the rest of the paper's numbers use (20 B). The second projects the full §8.2.2 spec surface — adding the SQ/RQ/JFC/JFAE handle table, the state-machine bookkeeping for Suspend-mode drain, exception-mode and fault counters, the public-Jetty owner field, and the Jetty-Group back-pointer (§8.2.2.1 Type 3). The full-spec Jetty grows from 20 B to 48 B; the asymptotic ratio at (1024, 1024) shifts from 4,855 $\times$ to 3,855 $\times$. Both numbers are orders of magnitude away from the RoCE per-QP regime, so the framing of "Pillar 1 = state-byte scaling" survives the honest accounting — the residual 200 B of spec-surface fields the MVP elides do *not* explain the gap.

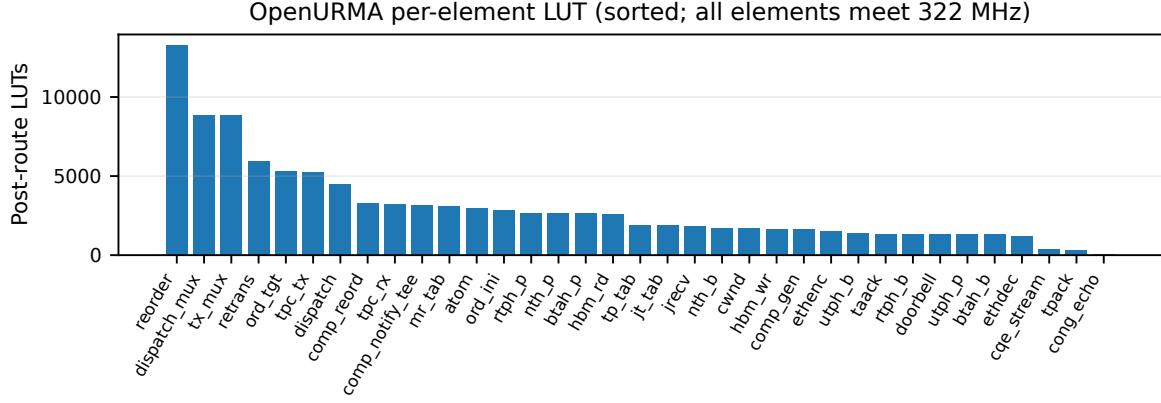


Figure 26: Per-element post-route LUT, sorted descending. All 38 OpenURMA elements meet 322 MHz with positive WNS. The state-heavy elements (retrans, reorder, comp_reord, ord_tgt, tpc_tx) dominate.

Table 2: Per-connection state, decomposed against UB-Base-Specification 2.0.1 fields. Per-Jetty columns: MVP = today’s UB_Jetty_Table : : JT; full-spec = projected after adding state-machine drain, exception-mode, public-Jetty owner, and Jetty-Group back-pointer. Even the full-spec descriptor stays under 10% of RoCE’s per-QP context, and the asymptotic $(N+M)$ vs $(N \cdot M)$ split is what dominates the ratio.

Jetty (MVP)	B	Jetty (full §8.2.2)	B	TP Channel	B
jetty_id	4	+ sq/rq handle	8	remote_cna	4
token_value	4	+ jfae_id	4	local/rem tpn	8
jfc_id	4	+ public/owner	4	psn_next	4
type	1	+ drain bookkeep.	6	tpmsn_next	4
state	1	+ exc. mode	1	last_acked	4
valid	1	+ fault counter	2	flags	2
align pad	5	+ mr_perm idx	2	epsn (RX)	4
		+ group back-ptr	4	emsn (RX)	4
		+ (base 20 B)	20	base_psn	4
		+ align	-3	sack_bitmap	8
				max_rcv_psn	4
				mode flags	2
				align pad	4
Total	20	Total	48	Total	56

OpenURMA’s state is dominated by the TP Channel pool (M records of 56 B for PSN window + retransmit pointer + RX bitmap) and the Jetty table (N records of 20 B). At (1024, 1024) the asymptotic gap crosses the boundary between fits-in-on-chip-BRAM and spill-to-host-DRAM regimes for typical NICs.

10.4 Headline trade-off

At $(N, M) = (1024, 1024)$:

- OpenURMA holds $4,855\times$ less per-connection NIC state.
- OpenURMA holds $20.8\times$ less host-side memory in the user-space verb library (24.2 MB vs 504 MB).
- OpenURMA delivers $2.80\times$ higher sustained throughput (150.36 vs 53.62 WR/ μ s, paced microbench; 159.46 vs 53.65 WR/ μ s under 1000-WR burst saturation).
- OpenURMA pays $2.63\times$ more LUT, $2.11\times$ more FF, $4.90\times$ more BRAM18 (the BRAM growth comes from

(N, M)	OpenURMA	OpenRoCE	Ratio
(1, 1)	108 B	544 B	$5.0\times$
(8, 8)	864 B	33 KB	$38\times$
(64, 64)	6.7 KB	2.0 MB	$304\times$
(256, 256)	27 KB	33 MB	$1,214\times$
(1024, 1024)	110.6 KB	536.9 MB	$4,855\times$
(1024, 1024) full-spec Jetty	139.3 KB	536.9 MB	$3,855\times$

the multi-flit-aware queues in jsched/ord_ini that hold up to 12 flits per packet).

- OpenURMA’s TX pipeline is 24 cycles cold-start (74.54 ns at 322 MHz); OpenRoCE’s measured 9 cycles (27.95 ns). The 46.6 ns delta is the cost of the longer pipeline.

For RDMA workloads where NIC state is the binding resource (HPC all-to-all, AI training collectives), OpenURMA’s design point is the clear win: the silicon area cost is bounded ($\approx 14\%$ of a U50) and the latency cost is small relative to network RTT, while the state saving crosses orders of magnitude. Pillar 2 (the gating elements) costs only 13 KLUT of the 123 KLUT total — 10.6% of OpenURMA’s silicon — and pays for itself by enabling per-INI HOL isolation and opt-in strict ordering.

11. Discussion and Limitations

Out-of-context vs platform-shell synthesis. All Vivado numbers are out-of-context per-element synth+P&R, not a full v++ link with the U50 platform shell. Adding the platform shell adds $\sim 50\text{--}80\text{K}$ LUTs of fixed overhead identical for both stacks (CMAC + XDMA + NoC), which would not change the OpenURMA-vs-OpenRoCE ratios. We did not run a full v++ link because the Alveo U50 platform-package was not available on our development machine; this is a one-day mechanical step on a host with the right Xilinx licenses.

No silicon yet. Everything in this paper is HLS-estimated + Vivado-measured area / timing. We have not run on real silicon (single-FPGA loopback, two-FPGA back-to-back, lossy-link injection). The 322 MHz numbers are

post-route static-timing-analysis; we have high confidence those hold in silicon at the U50's -2 speed grade, but we have not measured wire-time first-message latency under loss or contention. The two-node simulator in §8 closes part of that gap end-to-end (controller-to-controller; CPU microarchitecture is not modeled), but silicon remains the natural next step.

Out-of-context routing optimism. Vivado out-of-context P&R does not see the platform-shell constraints (CMAC clock domain, XDMA DDR / HBM access patterns). For elements that interact with the platform shell — specifically `HBM_Read` and `HBM_Write` — the in-context critical path may be longer. Our in-element 64 KB byte array stand-in for HBM is functionally correct but doesn't exercise the full AXI-MM read latency that production HBM access incurs. The intended FPGA wiring uses the `UBFPGA_HBM_*` variants (`UBFPGA_HBM_Read.clnp`, `UBFPGA_HBM_Write.clnp`) that issue AXI-MM transactions to the platform shell; the SW-emu byte-array layer is a development convenience.

No driver / kernel module. `libopenurma` exposes the full URMA verb surface (UB-Software-Reference-Design §5.3), including pre-posted RQEs. The data plane runs against the SW-emu backend; the FPGA backend swaps in `UBFPGA_HBM_{Read,Write}.clnp` (which emit `#pragma HLS INTERFACE m_axi` for the platform shell's HBM ports), but the kernel-module / IOCTL surface that rings the PCIe doorbell is not implemented — we have no FPGA in the loop yet.

Out-of-scope: collective offload and URMA-CTP. Two UB capabilities present in the Ascend 950 silicon [13] are deliberately out of scope here. (i) The 950 ships a *Collective Communication Unit* (CCU) that executes Broadcast / Reduce-Scatter / All-Gather / All-Reduce / All-to-All directly in hardware on top of URMA; OpenURMA exposes the underlying URMA verbs but not a collective engine — workloads that want collectives drive them from software through the verb library. (ii) The 950 distinguishes *URMA-CTP* (Compact Transport; reliability delegated to the lower layer) from *URMA-TP* (full end-to-end reliable transport) as named transport modes; OpenURMA implements only the RTP-equivalent path. The TPG/spray multi-path element (§3) lowers a UNO+NO workload into something behaviourally close to CTP — no per-INI ordering, no in-buffer reordering — but exposing CTP as an explicit transport mode with its own opcode set is a natural follow-on item.

§8.2.2 spec-surface gaps that remain. `UB_Jetty_Group` (§3.6) closes the single largest §8.2.2 omission flagged in earlier drafts and quantified in §8.10, but several spec features remain MVP. The per-Jetty state machine (Reset / Ready / Suspend / Error, §8.2.2.3) carries a state byte but no transition logic for recoverable-fault SQ drain; exception-mode Continue

vs Suspend is not wired through; the JFAE async-event queue and the reserved public-Jetty IDs (0–31, §8.2.5) are not implemented; and token-based access control at the RX Jetty boundary is enforced at MR granularity only, not per Jetty. Table 2 projects the per-Jetty byte cost of closing these (20 B → 48 B; asymptotic ratio shifts from 4,855× to 3,855×, so the Pillar 1 framing is robust). The Jetty-Group RTL itself has not yet been independently P&R'd; it follows the same $\Pi=2$ pattern as the four other gating elements (`ord_ini`, `ord_tgt`, `comp_reord`, `jsched`, all of which close 322 MHz at $\Pi=2$ per §4) and is in-line with §10's 13 KLUT ordering category as a follow-on item.

Comparison framing. OpenRoCE is an apples-to-apples baseline, not an optimised production stack. It anchors the comparison on identical infrastructure (same toolchain, same target, same harness). We do not address “how does OpenURMA compare to ConnectX-7?” — that would require disentangling the protocol design from a vendor's many-product-cycle optimisation budget. We address “how do UB's three pillars trade against the standard RoCE design point in synthesisable RTL on identical infrastructure?”

Why no SRNIC / StaR / 1RMA / Falcon / UEC in-fabric baselines. SRNIC [45], StaR [44], Aquila [7], and Falcon [41] are vendor-internal silicon; 1RMA [40] is a software stack atop commodity RoCE; UEC [43] is a specification without public RTL; and the IRN/MELO/-FaSR/DCP [34, 32, 10, 29] SR families are algorithm-and-simulation or vendor-internal hardware studies. None reduces to synthesisable FPGA RTL without effectively re-implementing it. We treat these as architectural reference points (§12, Table 3) rather than numerical baselines, and use OpenRoCE as the single in-fabric baseline because it is the only design point where every variable below the protocol can be held fixed.

12. Related Work

RDMA scalability has become one of the most active areas in datacenter networking, with substantial work across NSDI / SIGCOMM / OSDI / ATC in 2020–2025. We survey the field along five orthogonal axes and summarise OpenURMA's positioning in Table 3.

Connection-state scalability. The QP-scaling problem has driven a long line of work. FaSST [15] reduced the per-QP footprint via a UD verbs layer over fast packet processing. 1RMA [40] dropped per-connection state entirely by issuing one-sided RDMA's over UDP with explicit per-op authentication. Collie [20] characterised the RoCE-on-Ethernet performance cliff under PFC oscillation. SRNIC [45] proposed a connection cache atop a fixed-size on-chip QP context, spilling cold connections to host memory under hardware control. StaR [44] pushes the same axis further by reconstructing per-connection state on demand

Table 3: Design-point positioning vs. recent RDMA scalability work (2018–2025). Rows are grouped by primary contribution. “Conn. state” is what the NIC stores per peer pair / per connection; “Loss recov.” is the on-NIC loss-recovery scheme; “Multi-path” is whether the transport admits multi-path delivery without reorder breakage; “Ordering surface” is what semantic ordering the spec exposes; “Substrate” is the realisation. UB/OpenURMA is the only point that combines per-host-pair connection state, a graded §7.3 ordering surface, and an open FPGA-RTL substrate.

Work	Conn. state	Loss recov.	Multi-path	Ordering surface	Substrate
RoCEv2 RC	per-QP	GBN	–	strict / QP	vendor ASIC
FaSST [15]	UD (per msg)	app-level	–	none	SW + commodity RNIC
IRMA [40]	none	per-op auth	yes	none	SW + RoCE
Aquila + IRMA [7]	cell, none	cell-level	cell-network	none	custom ASIC
SRNIC [45]	QP cache + spill	RoCE	–	strict / QP	vendor ASIC
StaR [44]	reconstructed	RoCE	–	strict / QP	vendor ASIC
Falcon [41]	per-conn	SACK + RTT	yes	per-flow	vendor ASIC
UEC v1.0 [43]	packet-sprayed	SACK	yes	per-transaction	industry spec
Tonic [1]	programmable	programmable	programmable	programmable	FPGA (Verilog)
NanoTransport [14]	programmable	programmable	programmable	programmable	FPGA (Chisel/P4)
StRoM [39]	per-QP	RoCE-derived	–	per-QP	FPGA (HLS)
Flor [28]	per-QP	RoCE	–	per-QP	open framework
SwCC [9]	per-QP	RoCE + SW CC	–	per-QP	NIC + RISC-V
IRN [34]	per-QP	SR + per-pkt ACK	–	strict / QP	RoCE delta
MELO [32]	per-QP	const-mem SR	–	strict / QP	algorithm + sim
FaSR [10]	per-QP	line-rate SR	–	strict / QP	RNIC
DCP (SIGCOMM’25) [29]	per-QP	RTO-free SR	yes (pkt-LB)	strict / QP	hardware
LEFT [11]	per-QP	shared bitmap	yes	strict / QP	RNIC + sim
MP-RDMA [33]	per-QP + OOO	RoCE	yes	OOO + bitmap	RoCE delta
ConWeave [42]	per-QP	RoCE	yes	per-QP	switch + NIC
STrack [21]	per-flow	SR + fast recov	yes	per-flow	hardware
Spectrum-X [37]	per-QP	RoCE	yes	per-QP	NIC + switch
Justitia [46]	per-QP	–	–	–	SW userspace
Husky [19]	per-QP (diag)	–	–	–	test suite
Harmonic [31]	per-QP	–	–	–	hardware (BF-3)
SCR [47]	SW-programmable	SW-prog	SW-prog	SW-prog	BlueField-3 + DPA
UB / OpenURMA	per host-pair	GBN + TPSACK	TPG + spray	graded §7.3	open FPGA RTL

from host memory and packet metadata, so the NIC carries effectively zero per-QP context in steady state. Meta’s production deployment at SIGCOMM’24 [6] gives the empirical face of the same problem: their distributed-training backend needs up to 32 QPs per source–destination pair to approach roofline throughput, exactly because RoCE RC’s per-QP state pins each QP to a small set of paths. UB attacks the problem at the transport-layer *specification*, by making the connection a property of the host pair (TP Channel) rather than the application pair (QP); this is complementary to SRNIC’s cache and StaR’s reconstruction, since collapsing the cardinality of per-pair state also bounds whatever those techniques would otherwise have to spill or rebuild.

New reliable hardware transports. Aquila [7] co-designed a custom fabric with IRMA cells to extend memory accesses across a clique without a host-pair connection. Google’s Falcon [41], contributed to OCP in 2024 and presented at SIGCOMM’25, retains a per-connection abstraction but adds hardware SACK, fine-grained RTT-based shaping, and PSP-encrypted multi-path; it claims a $5 \times$ op-rate and $10 \times$ tail-latency improvement over the Pony Express software transport, and outperforms RoCE on its target workloads. Ultra Ethernet Specification 1.0 [43], released June 2025 by a >100 -company Linux Foundation consortium, defines a packet-sprayed, SACK-based reliable transport with native RDMA semantics and scales to “millions of endpoints”. UB is in the same architectural family — a clean-slate, connectionless, RDMA-class transport

designed for AI/HPC workloads — but with a different ordering surface (graded §7.3 vs. UEC’s per-transaction guarantees) and an explicitly open spec and (with OpenURMA) open RTL. The first publicly disclosed silicon implementing UB itself is Huawei’s Ascend 950 NPU [13], which ships URMA-CTP / URMA-TP transport modes, a UB-Memory synchronous Load/Store + atomic path that mirrors §8.3, a UB On-Chip Switch for in-die forwarding, UBoE escape to Ethernet, and a hardware Collective Communication Unit on top of URMA — targeting an 8192-card UB super-node scaling to $>128K$ -card clusters, the deployment regime where the $O(N+M)$ -vs- $O(N \cdot M)$ gap quantified in §10 becomes operationally binding rather than asymptotic. The Ascend 950 disclosure is an architectural reference point, not a numerical baseline: the silicon is closed and reports no per-element area or per-verb latency that would map onto our FPGA harness.

Programmable and FPGA-based transports. Tonic [1] introduced a Verilog-programmable transport-protocol framework on FPGA capable of expressing TCP, RoCE, and NDP variants in a unified pipeline. NanoTransport [14] provided a P4-programmable hardware transport prototype in Chisel/Firesim, with NDP and Homa demonstrations. StRoM [39] put a fully programmable RoCE-derived path on FPGA. Flor [28] is an open RDMA framework over heterogeneous RNICs. SwCC [9] adds an on-NIC RISC-V engine for software-programmable, per-packet congestion control. NICA [4] and AccelNet [5] build NIC offloads on programmable hardware. ClickNP [25]

is the element-based FPGA substrate we build on through OpenClickNP [22], and KV-Direct [24] is a precedent for ClickNP-style decomposition applied to a specific RDMA workload. OpenURMA’s contribution in this category is the first end-to-end UB data-path on commodity FPGA closing P&R at 322 MHz, not the FPGA-programmability story itself — our substrate is fixed-function once compiled.

Loss recovery and selective retransmission. IRN [34] is the canonical argument that an RDMA NIC should abandon Go-Back-N in favour of bitmap-tracked selective retransmission with per-packet ACKs, removing much of RoCE’s reliance on PFC. MELO [32] sized the on-chip book-keeping for scalable SR, using a shared bits pool to keep SACK metadata bounded regardless of connection count. FaSR [10] drives full-line-rate SR on the NIC with novel state-management schemes addressing the connection-vs-memory tension. DCP [29], the SIGCOMM’25 Best Student Paper Honorable Mention, presents an RTO-free, packet-LB-friendly hardware-offloadable SR design. LEFT [11] optimises packet reordering with a shared-bitmap pool and packet aggregation. OpenURMA’s transport path implements both GBN and a selective TPSACK-driven retransmit slot (§3.4); we treat the above as algorithmic reference points, and a quantitative comparison under controlled loss injection (goodput-vs-loss-rate and retrans-buffer-occupancy curves) is left as follow-on work consistent with the SW-emu loss-injection gap flagged in §6.

Multi-tenant performance isolation. A second scalability axis — orthogonal to per-pair state — is isolation between cotenants sharing the same NIC. Justitia [46] provides software multi-tenancy via userspace pacing and rate-limiting. Husky [19] characterises how RNIC microarchitecture resources (MTT/MPT cache, processing units) can be exhausted by adversarial workloads, showing SR-IOV and HW TC do not isolate at the microarchitectural level. Harmonic [31] adds hardware-assisted isolation for public-cloud RDMA. OpenURMA does not currently address this axis; UB’s per-host-pair connection model nevertheless removes one of the cache-pressure sources Husky exploits, since the cache no longer scales with application-pair count.

Multi-path and adaptive routing. MP-RDMA [33] first observed that letting RDMA WRITE/READ packets traverse multiple paths and arrive out-of-order, with bitmap-tracked completion, recovers substantial multi-path bandwidth without breaking the workloads that consume it. ConWeave [42] extends this into the switch with in-network reordering support for RoCE. STrack [21] is a hardware-offloaded multipath transport tuned for AI/ML clusters, combining adaptive load balancing, RTT-based CC, and SR-based loss recovery. Spectrum-X [37] adds adaptive-routing multi-path to commercial RoCE-on-Ethernet NICs. UB’s TPG/spray [12] reuses the same architectural idea — spray flits across the path-diversity hash, recover order at the receiver — exposed as the default mode of the graded §7.3 ordering surface (UNO + NO); we implement TPG +

spray in the UB_TPG_Group element (§3).

Software-controlled hardware transport. SCR / White-Boxing RDMA [47] is a complementary recent direction: it surfaces packet-granular software control over the BlueField-3 hardware transport via on-NIC Datapath Accelerators, enabling SW-defined CC, scheduling, and multi-path on top of vendor RoCE silicon. OpenURMA targets a different trade — the entire transport is RTL on FPGA, with no SW fast-path involvement — but the two are not mutually exclusive: a future revision could expose a similar SW-programmable surface on top of OpenURMA’s open RTL.

Datacenter transports and congestion control. DC-QCN [48] is the de-facto RoCE congestion control; we mirror it as the OpenRoCE baseline’s CC. UB’s C-AQM [12] is a queue-occupancy-based ECN signal; we model it as a SystemC + SW-emu element so the closed-loop behaviour is testable end-to-end without a physical switch.

Ordering semantics: total order vs. relaxed order. At the opposite end of the ordering design space from URMA, 1Pipe [27] provides a causally and totally ordered communication abstraction across a whole datacenter, with the goal of simplifying distributed transactions, state-machine replication, and distributed atomic operations. 1Pipe pays for the guarantee with in-network barrier aggregation at every switch and an extra ~ 0.5 – 1 RTT of barrier-wait latency on each message, with reliable scatterings adding a further RTT for 2PC commit. URMA traffic is the wrong workload for that trade. The bulk of URMA verbs — ML training collectives, KV-cache / tensor transfers, parameter-server fan-in/fan-out, distributed checkpoint — are bandwidth-bound memory operations whose correctness contract is scoped to a single (Initiator, target Jetty) pair; cross-host total order is not part of the contract and paying for it costs real RTTs on the critical path. URMA’s §7.3 graded surface (§2.3) is the specification-level descendant of MP-RDMA’s relaxed-ordering insight discussed above: UNO + NO is the default fast path (no per-INI ordering at all, the regime where MP-RDMA’s multi-path delivery is safe by construction), ROI / ROT / ROL graduate strictly more ordering as the workload demands it, and SO + Fence is the escape hatch for the rare verb that does need strict per-WR serialisation. 1Pipe-style total order remains a good fit for the higher-level distributed-transaction layer that may sit on top of URMA — not for the verb path itself, which is what this paper implements.

Adjacent fabrics. NVLink [35] and NVLink Fusion are NVIDIA-internal fabric protocols whose specification is partially open. CXL [3] is a load/store fabric over PCIe physical layers. The standard framing treats CXL and UB as different niches (intra-rack memory-coherence vs. scale-out DC-network); a more accurate framing is that they are different layers of the same architectural axis. CXL is the physical+link-layer mechanism for memory-semantic access at intra-chassis distances; UB’s §8.3 Load/Store + §6.1

TP Bypass is the transport-layer mechanism for memory-semantic access at arbitrary cross-chassis distances. They are complementary rather than competing: a future system could route Load/Store to CXL while in-chassis and to UB across the rack, with the same ISA instruction reaching either substrate without an API change. UB’s transport-layer contribution is precisely what permits the extension — a load/store semantic that does not terminate at the PCIe boundary the way CXL does.

System-layer scaling and the case for memory-semantic fabrics. He [8] frames UB as the system-layer instance of a broader “ τ scaling” principle: with geometric scaling’s historical dividends flattening past the 7 nm node, the unifying optimization target across the stack becomes the characteristic time constant τ at each layer (transistor, circuit, chip, system), and the system-layer mechanism for compressing τ is to collapse the conversion-bound multi-protocol stack (PCIe $\rightarrow$ chassis fabric $\rightarrow$ Ethernet/IB $\rightarrow$ SW RDMA) into a single memory-semantic peer-to-peer fabric. The paper reports an empirical $\sim 500\times$ end-to-end latency reduction for UB’s ASIC deployment from ~ 10 s of μ s (TCP/IP-class) to ~ 100 ns. OpenURMA is the public open-source artifact of the protocol layer in that argument: §7 quantifies the cycle-accurate NIC pipeline (24 cycles cold-path on the URMA-async path, 8 cycles on the §8.3 Load/Store + TP Bypass path) and §8 quantifies the end-to-end latency budget in a two-node controller-to-controller simulator (500 ns UB Load/Store vs 2186 ns RoCE-DMA at 100 ns link delay on a 64 B READ, controller-side only — no CPU-microarchitecture modelling). Our absolute floor on FPGA at 322 MHz is therefore in the same order of magnitude as He’s ~ 100 ns ASIC end-to-end target, and the architectural mechanism — layer collapse via memory-semantic transport — is the one He identifies.

13. Conclusion

OpenURMA puts UB’s three architectural pillars — per-host-pair connection state, a graded ordering surface, and a memory-semantic §8.3 Load/Store data path through an on-chip-bus-attached controller — into synthesisable RTL on a commodity FPGA. All 38 elements close 322 MHz Vivado P&R; the 21-element OpenRoCE baseline closes the same target on the same toolchain.

Three observations. **Pillar 1 (scalability):** the per-host-pair connection model converts $N \cdot M$ state into $N + M$ growth. The gap is asymptotic, not constant: $5\times$ at (1, 1), $4,855\times$ at (1024, 1024), crossing the on-chip-BRAM-to-host-DRAM spill boundary between $N=23$ and $N=1024$. **Pillar 2 (ordering):** 13 KLUT (10.6% of the design) and zero pipeline cycles on paths that do not request ordering. The cost is paid only by workloads that opt in. **Pillar 3 (latency):** 8 cycles (≈ 25 ns) first-flit on the §8.3 path; ≈ 500 ns end-to-end CPU-to-remote-DRAM-to-CPU on a 64 B READ in the two-node simulator — $4.37\times$ below the matched RoCE-DMA baseline (2186 ns). The saving is structural: the nine PCIe-bound cost components

RoCE must pay each round-trip (§8.5) are the protocol consequence of disjoint CPU/NIC address spaces, and §8.3 removes the disjointness. **Combined price:** $\sim 14\%$ of a U50’s LUT budget, 46.6 ns of pipeline depth above the matched RC reference.

Synthesis. The three pillars do not stack independently. Pillar 3’s on-chip controller is feasible only because Pillar 1 has bounded the NIC’s on-chip context to ~ 100 KB. Pillar 2’s ~ 13 KLUT cost is structurally cheap only because Pillar 1 has already provisioned the per-Initiator sequence-number machinery the four modes select among. And the multi-path spray Pillar 1 authorises under UNO + NO is correctness-safe only because OpenURMA separates transport-layer PSN reordering from transaction-layer completion reordering. The single architectural commitment underneath is a memory-semantic peer-to-peer fabric in which connection state, ordering surface, and submission path are co-designed against the on-chip bus rather than retrofitted onto a PCIe-attached NIC abstraction. OpenURMA’s open RTL makes that commitment inspectable cell-by-cell; its $\sim 14\%$ U50 footprint says it is implementable on commodity silicon.

The artefact is intended as a substrate for further work — loss-recovery policies under multi-flit retransmission, alternative congestion signals, multi-path scheduling — and as a controlled baseline against which Falcon, UEC, and SRNIC/StaR-derivative transports can be compared on identical infrastructure.

Code. <https://github.com/bojieli/OpenURMA>.

Acknowledgments. This work builds on the OpenClickNP toolchain (an open re-implementation of the ClickNP element model) and the published UB-Base-Specification 2.0.1.

References

- [1] Mina Tahmasbi Arashloo, Alexey Lavrov, Manya Ghobadi, Jennifer Rexford, David Walker, and David Wentzlauff. Enabling programmable transport protocols in high-speed NICs. In *Proc. USENIX NSDI*, 2020.
- [2] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proc. ACM SoCC*, 2010. The Yahoo! Cloud Serving Benchmark; we use the YCSB-A 50/50 Get-Put Zipfian workload in §9.8.
- [3] CXL Consortium. Compute Express Link (CXL) Specification 3.1. <https://www.computeexpresslink.org/>, 2024.
- [4] Haggai Eran, Lior Zeno, Maroun Tork, Gabi Malka, and Mark Silberstein. NICA: An infrastructure for inline acceleration of network applications. In *Proc. USENIX ATC*, 2019.

- [5] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, et al. Azure Accelerated Networking: SmartNICs in the public cloud. In *Proc. USENIX NSDI*, 2018.
- [6] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. RDMA over Ethernet for distributed training at meta scale. In *Proc. ACM SIGCOMM*, 2024.
- [7] Dan Gibson, Hema Hariharan, Eric Lance, Moray McLaren, Behnam Montazeri, Arjun Singh, Stephen Wang, Hassan M. G. Wassel, Zhehua Wu, Sunghwan Yoo, Raghuraman Balasubramanian, Prashant Chandra, Michael Cutforth, Peter Cuy, David Decotigny, Rakesh Gautam, Alex Iriza, Milo M. K. Martin, Rick Roy, Zuowei Shen, Ming Tan, Ye Tang, Monica Wong-Chan, Joe Zbiciak, and Amin Vahdat. Aquila: A unified, low-latency fabric for datacenter networks. In *Proc. USENIX NSDI*, 2022.
- [8] Tingbo He. A time scaling theory for multi-layer electronic systems. *ChinaXiv*, May 2026. chinaxiv-202605.00224. Perspective from Huawei Semiconductor: τ scaling as successor to geometric Moore’s-Law scaling; positions Unified Bus as the system-layer τ reduction mechanism with end-to-end remote-access latency from ~ 10 s of μ s (TCP/IP-class) to ~ 100 ns.
- [9] Hongjing Huang, Jie Zhang, Xuzheng Chen, Ziyu Song, Jiajun Qin, and Zeke Wang. SwCC: Software-programmable and per-packet congestion control in RDMA engine. In *Proc. USENIX ATC*, 2025.
- [10] Peihao Huang, Guo Chen, Xin Zhang, Can Liu, Hongyu Wang, Huijun Shen, Ying Bian, Yuanwei Lu, Zhenyuan Ruan, Bojie Li, Jiansong Zhang, Yongfeng Liu, and Zhigang Chen. Fast and scalable selective retransmission for RDMA. In *Proc. IEEE INFOCOM*, 2025.
- [11] Peihao Huang, Xin Zhang, Zhigang Chen, Can Liu, and Guo Chen. LEFT: Lightweight and fast packet reordering for RDMA. In *Proc. APNet*, 2024.
- [12] Huawei Technologies. UB-base-specification 2.0.1. <https://www.unifiedbus.org/>, 2024. Unified Bus consortium specification, available from the consortium’s documentation portal.
- [13] Huawei Technologies. Ascend 950 NPU architecture white paper. Huawei vendor white paper, May 2026. Architectural disclosure for the Ascend 950PR and 950DT NPUs; first publicly documented silicon implementing the Unified Bus spec, with URMA (asynchronous Write/Read/Send/Atomic via Jetty) and UB Memory (synchronous Load/Store + AtomicStore/Load/Swap/CAS) exposed as distinct on-die paths, plus URMA-CTP / URMA-TP transport modes, UBoE 2×400 Gbps, a hardware Collective Communication Unit (CCU), UB On-Chip Switch for in-die forwarding, and a UB super-node target of 8192 cards (cluster target $> 128K$).
- [14] Stephen Ibanez, Alex Mallery, Serhat Arslan, Theo Jepsen, Muhammad Shahbaz, Nick McKeown, and Changhoon Kim. NanoTransport: A low-latency, programmable transport layer for NICs. In *Proc. ACM SOSR*, 2021.
- [15] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using RDMA efficiently for key-value services. In *Proc. ACM SIGCOMM*, 2014.
- [16] Anuj Kalia, Michael Kaminsky, and David G. Andersen. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *Proc. USENIX OSDI*, 2016.
- [17] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Datacenter RPCs can be general and fast. In *Proc. USENIX NSDI*, 2019.
- [18] Antoine Kaufmann, Tim Stamler, Simon Peter, Naveen Kr. Sharma, Arvind Krishnamurthy, and Thomas Anderson. TAS: TCP acceleration as an OS service. In *Proc. EuroSys*, 2019. Reports detailed PCIe-class transaction latency decompositions used as parameter references in §8.
- [19] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R. Lebeck, and Danyang Zhuo. Understanding RDMA microarchitecture resources for performance isolation. In *Proc. USENIX NSDI*, 2023.
- [20] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. Colli: Finding performance anomalies in RDMA subsystems. In *Proc. USENIX NSDI*, 2022.
- [21] Yanfang Le, Rong Pan, Peter Newman, Jeremias Blendin, Abdul Kabbani, Vipin Jain, Raghava Sivaramu, and Francis Matus. STrack: A reliable multipath transport for AI/ML clusters. arXiv:2407.15266, 2024.
- [22] Bojie Li. OpenClickNP: a clean-room reimplementa-tion of ClickNP on Alveo U50. <https://github.com/bojieli/OpenClickNP>, 2025–2026.
- [23] Bojie Li, Tianyi Cui, Zibo Wang, Wei Bai, and Lintao Zhang. SocksDirect: Datacenter sockets can be fast and compatible. In *Proc. ACM SIGCOMM*, 2019.
- [24] Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proc. ACM SOSP*, 2017.

- [25] Bojie Li, Kun Tan, Layong Larry Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high-performance network processing with reconfigurable hardware. In *Proc. ACM SIGCOMM*, 2016.
- [26] Bojie Li, Zhilong Xiang, Xiang Wang, Hongru Jonathan Zhou, and Kun Tan. FastWake: Revisiting host network stack for interrupt-mode RDMA. In *Proc. APNet*, 2023.
- [27] Bojie Li, Gefei Zuo, Wei Bai, and Lintao Zhang. lPipe: Scalable total order communication in data center networks. In *Proc. ACM SIGCOMM*, 2021.
- [28] Qiang Li, Yixiao Gao, Xiaoliang Wang, Haonan Qiu, Yanfang Le, Derui Liu, Qiao Xiang, Fei Feng, Peng Zhang, Bo Li, Jianbo Dong, Lingbo Tang, Hongqiang Harry Liu, Shaozong Liu, Weijie Li, Rui Miao, Yaohui Wu, Zhiwu Wu, Chao Han, Lei Yan, Zheng Cao, Zhongjie Wu, Chen Tian, Guihai Chen, Dennis Cai, Jinbo Wu, Jiaji Zhu, Jiesheng Wu, and Jiwu Shu. Flor: An open high performance RDMA framework over heterogeneous RNICs. In *Proc. USENIX OSDI*, 2023.
- [29] Wenxue Li, Xiangzhou Liu, Yunxuan Zhang, Zihao Wang, Wei Gu, Tao Qian, Gaoxiong Zeng, Shoushou Ren, Xinyang Huang, Zhenghang Ren, Bowen Liu, Junxue Zhang, Kai Chen, and Bingyang Liu. Revisiting RDMA reliability for lossy fabrics. In *Proc. ACM SIGCOMM*, 2025. Best Student Paper, Honorable Mention.
- [30] Xiaofeng Lin, Yu Chen, Xiaodong Li, et al. Fast-socket: An almost drop-in replacement for the Linux socket interface for High-Performance Networking. In *Proc. USENIX ATC*, 2017.
- [31] Jiaqi Lou, Xinhao Kong, Jinghan Huang, Wei Bai, Nam Sung Kim, and Danyang Zhuo. Harmonic: Hardware-assisted RDMA performance isolation for public clouds. In *Proc. USENIX NSDI*, 2024.
- [32] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Memory efficient loss recovery for hardware-based transport in datacenter. In *Proc. APNet*, 2017.
- [33] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. Multi-Path transport for RDMA in datacenters. In *Proc. USENIX NSDI*, 2018.
- [34] Radhika Mittal, Alexander Shpiner, Aurojit Panda, Eitan Zahavi, Arvind Krishnamurthy, Sylvia Ratnasamy, and Scott Shenker. Revisiting network support for RDMA. In *Proc. ACM SIGCOMM*, 2018.
- [35] NVIDIA Corporation. NVLink: A high-bandwidth inter-GPU interconnect. Vendor whitepaper, 2014–2025. Successive generations of the NVLink fabric are described in the NVIDIA whitepaper series.
- [36] NVIDIA Corporation. NVIDIA BlueField-3 DPU datasheet. NVIDIA Networking product brief, 2023. Available from NVIDIA’s data-processing-unit product page.
- [37] NVIDIA Networking. NVIDIA Spectrum-X: Adaptive routing and telemetry-based congestion control for AI networks. NVIDIA technical brief, 2024. Vendor description of multi-path adaptive-routing delivery over Spectrum-4 / BlueField-3 NICs; the closest commercially-deployed point of comparison to UB’s TPG multi-path scheme.
- [38] Sebastian Ramos and Torsten Hoeffler. Designing high-performance, low-latency multi-cluster communication on modern InfiniBand networks. In *Proc. ACM HPDC*, 2023. Reports ConnectX-7 PCIe round-trip latencies in the ~300–500 ns range; we use this as the parameterised PCIe RTT in §8.
- [39] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *Proc. EuroSys*, 2020.
- [40] Arjun Singhvi, Aditya Akella, Dan Gibson, Thomas F. Wenisch, Monica Wong-Chan, Sean Clark, Milo M. K. Martin, Moray McLaren, Prashant Chandra, Rob Cauble, et al. 1RMA: Re-envisioning remote memory access for multi-tenant datacenters. In *Proc. ACM SIGCOMM*, 2020.
- [41] Arjun Singhvi, Nandita Dukkkipati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyojeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. Falcon: A reliable, low latency hardware transport. In *Proc. ACM SIGCOMM*, 2025. Specification contributed to OCP 2024.
- [42] Cha Hwan Song, Xin Zhe Khooi, Raj Joshi, Inho Choi, Jialin Li, and Mun Choon Chan. Network load balancing with in-network reordering support for RDMA. In *Proc. ACM SIGCOMM*, 2023.
- [43] Ultra Ethernet Consortium. Ultra Ethernet specification 1.0. Industry specification, 2025. Released June 2025 under Linux Foundation JDF; <https://ultraethernet.org/>.
- [44] Xizheng Wang, Guo Chen, Xijin Yin, Huichen Dai, Bojie Li, Binzhang Fu, and Kun Tan. StaR: Breaking the scalability limit for RDMA. In *Proc. IEEE ICNP*, 2021.

- [45] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, Xinchun Wan, Peng Xie, Tao Feng, Ke Cheng, Xiongfei Geng, Tianhao Wang, Weicheng Ling, Kejia Huo, Pingbo An, Kui Ji, Shideng Zhang, Bin Xu, Ruiqing Feng, Tao Ding, Kai Chen, and Chuanxiong Guo. SRNIC: A scalable architecture for RDMA NICs. In *Proc. USENIX NSDI*, 2023.
- [46] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. Justitia: Software multi-tenancy in hardware kernel-bypass networks. In *Proc. USENIX NSDI*, 2022.
- [47] Chenxingyu Zhao, Jaehong Min, Ming Liu, and Arvind Krishnamurthy. White-boxing RDMA with packet-granular software control. In *Proc. USENIX NSDI*, 2025.
- [48] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for Large-Scale RDMA deployments. In *Proc. ACM SIGCOMM*, 2015.